

Bridging Bitcoin to Second Layers via BITVM2

Robin Linus Woll
Stanford University
ZeroSync

Lukas Aumayr
University of Edinburgh
Common Prefix

Zeta Avarikioti
TU Wien
Common Prefix

Matteo Maffei
TU Wien

Andrea Pelosi
University of Pisa, University of Camerino, TU Wien

Orfeas Stefanos Thyfronitis Litos
Imperial College London, Common Prefix

Christos Stefo
TU Wien

David Tse
Stanford University, Byzantine Research

Alexei Zamyatin
BOB

Abstract

A holy grail in blockchain infrastructure is a trustless bridge between Bitcoin and its second layers or other chains. We make progress toward this vision by introducing the first light-client-based Bitcoin bridge. At its heart lies BITVM2-CORE, a novel paradigm that enables arbitrary program execution on Bitcoin, combining Turing-complete expressiveness with the security of Bitcoin consensus. BITVM2-BRIDGE advances prior approaches by reducing the trust assumption from an honest majority (t -of- n) to existential honesty (1-of- n) during setup. Liveness is guaranteed with only one rational operator, and any user can act as a challenger, enabling permissionless verification. A production-level implementation of BITVM2 has been developed, and a full challenge verification has been executed on the Bitcoin mainnet.

1 Introduction

Bitcoin, the largest cryptocurrency by market cap, has two major drawbacks: (i) a *low throughput* of about 5-7 transactions per second, limiting its scalability; and (ii) a *limited scripting language* (Bitcoin Script). While the latter significantly reduces the attack surface, it also restricts expressiveness, making Bitcoin ill-suited for decentralized applications such as DeFi. As a result, these applications typically run on blockchains with smart contract capabilities like Ethereum.

Limitations of the base blockchain (L1) can often be mitigated by adding second layers (L2s). For example, Bitcoin scalability has been addressed by L2 protocols such as payment channels [29]. Payment channels enable the Bitcoin asset to be transacted off the Bitcoin chain and settled back on Bitcoin in a completely trustless manner. However, payment channels serve to scale only one application: payments, Bitcoin’s original application. To make Bitcoin support a broader range of applications (e.g., DeFi) Bitcoin needs L2s with general-purpose capabilities. To that end, the holy grail is a *bridge* that allows Bitcoin assets to be trustlessly transferred back and forth between such an L2 and Bitcoin, where a wrapped version of the Bitcoin asset is used on the L2.

Honesty requirement	BitVM2 bridge	MultiSig bridge
Peg-in liveness	n-of-n signers 1-of-m operators	majority
Peg-in safety	1-of-n signers	majority
Peg-out liveness	1-of-m operators	majority
Peg-out safety	1-of-n signers 1 challenger	majority
Operator safety	1-of-n signers 1 challenger	N/A

Figure 1: Honest requirements of the BITVM2-BRIDGE as compared to a multi-signature bridge, e.g., [8, 26]. Peg-in refers to the transfer of bitcoins from the main chain to the L2. Peg-out refers to the transfer back to the main chain. Precise definitions of security properties are found in Appendix D.3.3. Security analysis is given in Section 6.

Trustless bridges between other blockchains are typically realized through *light clients* on the destination chain, which check if a certain transaction happened in the source chain before releasing tokens in the destination chain. The Inter-Blockchain Communication protocol (IBC) in Cosmos [21] is a notable example. Such light clients are typically implemented as smart contracts on the destination chains. Since Bitcoin lacks smart contracts, existing bridges typically use multi- or threshold signature schemes, where a group of t -of- n signers is entrusted with safekeeping BTC. If these signers collude or are bribed, user deposits are lost.

In this paper, we present the first light-client-based Bitcoin bridge, the BITVM2-BRIDGE. The BITVM2-BRIDGE is run by n permissioned signers, m permissioned operators, and an arbitrary number of *permissionless* challengers that watch over the operators. As shown in Fig. 1, BITVM2-BRIDGE achieves stronger security guarantees than standard multi-signature bridges. In particular, it reduces the peg-out safety and liveness assumptions from honest majorities to *existential honesty*: the presence of just one honest party per role (signer, operator, challenger) suffices.

The signers act as an emulation of a *covenant*, a set of pre-

signed transactions forming a contract on how and to whom the locked pegged-in funds can be released in a peg-out. Unlike in a multi-signature bridge, the signers only need to be present at peg-in, and once the transactions are presigned, they can delete their keys. The peg-out process is achieved in a trust-minimizing way through the light client. A user first burns the wrapped asset on the L2. Then one of the operators fronts the bitcoins to that user, and using a proof of burn, the operator claims the locked funds that were pegged in. The on-chain light client is used to verify the correctness of the proof and ensures that an honest operator can always claim the funds. Precisely because of the light client, no action by the signers is needed at peg-out, and as long as at least one of the signers deleted its key after peg-in, the covenant is sealed, and the peg-out process is trustless. This is the strong distinction between the BITVM2-BRIDGE and a standard multi-signature bridge.

At the heart of the light client lies BITVM2-CORE, the first permissionless protocol to compute arbitrary functions in Bitcoin, which we consider of independent interest. In particular, BITVM2-CORE allows an operator to assert that a Bitcoin Script program g executed on input x returns $y = g(x)$. The operator stakes a deposit of d BTC, where $d \geq 0$, and posts a commitment to this claim on-chain. If the claim is correct, they reclaim the deposit. Otherwise, any party can challenge the claim by executing a portion of the program on-chain and disproving the claim, preventing the operator from reclaiming their deposit. This is achieved by encoding a SNARK verifier in a sequence of Bitcoin transactions.

A production-level reference implementation of BITVM2-CORE has been developed. An experiment on Bitcoin mainnet shows that a worst-case (“unhappy-path”) dispute settles in under 8 hours and less than 0.15 BTC in fees, while 90 % of Disprove scripts remain below 3 MB. However, a careful incentive design ensures this path is never taken by rational parties, as operators risk losing their funds while malicious challengers have to cover the entire on-chain cost. The happy-path, which is the one normally executed, involves transactions that remain below 16.75 vkB, which is around 50k sat (roughly \$53). We discuss how to further reduce this cost to 534 vbytes, or around 1.6k sat (around \$1.66) in Section 8. This confirms BitVM2’s real world practicality.

Related work. A number of works approach the problem of bridging assets between different chains with minimal trust. Similarly to our work, zkBridge [34] leverages zkSNARKs to prove to one chain the state of another chain, thus achieving trustless bridging. Unfortunately, verifying zkSNARKs directly on Bitcoin is impossible due to its maximum block size, so zkBridge is incompatible with Bitcoin.

In parallel, several approaches to achieving arbitrary computation on Bitcoin have been proposed: BitVM [6], the predecessor of the current work, encodes arbitrary computation in a challenge-based protocol which requires 3 transactions on

chain in the optimistic case. In the case of dispute, a *bisection game* requires 79 sequential on-chain transactions, for a cost of approximately 0.007 BTC. In comparison, BITVM2-CORE requires fewer transactions, albeit larger. The main difference, however, is that BitVM is applicable in a permissioned, two-party setting, where prover and verifier are fixed in advance, and the verifier has to be online to punish the prover in case of misbehavior: this hinders a number of use cases, such as the bridge here proposed.

BitVMX [23] achieves a lower cost in terms of transactions compared to BitVM in exchange for requiring a permissioned set of operators. TOOP [12] extends BitVMX by moving funds to an address controlled by all active operators at the time of peg-out and having them send their private keys to the peg-out party. Unfortunately, its practicality is questionable as it needs off-chain communication exponential in the number of operators in the worst case.

ColliderVM [19] focuses on providing stateful computation on Bitcoin. It improves on BitVM by (i) permitting permissionless operators and by (ii) replacing the bisection game with executing stateful computation directly on-chain. Its main limitations are (i) the high energy requirements needed to find hash collisions and (ii) its need to trust a permissioned set of parties both for liveness and for safety as in BitVM.

The BITVM2-BRIDGE relies on a group of signers to emulate *covenants*: constraints on when UTXOs are spendable that are inherited by descendant UTXOs. True covenants can remove our reliance on existentially honest signers. Unfortunately, covenants are not available in today’s Bitcoin Script. As exemplified by [9], the few additional opcodes available in the Bitcoin fork BSV are enough to express covenants. Furthermore, PIPE [20] describes a way to simulate covenants on Bitcoin without a consensus change.

Real-world impact. BITVM2-BRIDGE has already influenced the industry: several startups, including BOB, Babylon, Citrea, Alpen, Bitlayer, and Fiamma, have emerged to build Bitcoin bridges and L2s based on this work. Each has introduced practical improvements and trade-offs, some of which are detailed in Section 8.

2 Model and Protocol Overview

We next describe the setting, assumptions, and actors in our construction, and provide a high-level overview of the BITVM2-BRIDGE. Our model includes two ledgers, the parties interacting with them, and the cryptographic and system assumptions we rely on.

Our main application is a trust-minimizing Bitcoin bridge. Along the way we develop two components of independent interest: (1) BITVM2-CORE, a protocol for verifying arbitrary computations on Bitcoin, and (2) a Bitcoin light client built on BITVM2-CORE. These serve as building blocks for the bridge and are analyzed formally in the Appendix.

2.1 Ledger and Network Model

We consider two ledgers: Bitcoin ($\mathcal{L}_A$) and a secondary ledger ($\mathcal{L}_B$). Both ledgers are assumed to satisfy the usual safety and liveness properties [13], i.e., honest parties’ views of each ledger are consistent (no two honest nodes confirm conflicting histories), and any transaction broadcast by an honest party will be confirmed within a bounded time.

We assume parties know each other’s identities and communicate via authenticated channels. We also assume messages are delivered within a bounded message delay Δ , i.e., we adopt a synchronous round-based model, as in the Bitcoin backbone framework [13]. Formal definitions can be found in Appendix D.

2.2 UTXO Model Overview

In Bitcoin, coins are modeled in the standard *unspent transaction output* (UTXO) paradigm: coins reside in transaction outputs, each carrying a value and a *locking script*. An output can be spent by providing a *witness* that satisfies its script. Transactions consume existing outputs as inputs and create new outputs, preserving value.

We rely on standard spending conditions used in Bitcoin, such as signature locks (CheckSig), multisignature locks (CheckMultiSig), time-based conditions (timelocks), and Taproot trees that enable complex, private policies (denoted $\langle \dots \rangle$), which we detail in Appendix B.2.

We additionally use spending conditions based on Lamport signatures, a one-time signature scheme verifiable on Bitcoin (cf. Appendix B.1). We leverage the scheme’s one-time security property to construct a commitment mechanism for messages (such as the intermediate states of a computation). In this construction, given a message m , the public key pk_m serves as its binding commitment instance, constraining the signer to a single future choice. The corresponding signature c_m represents the value selection: by revealing c_m , the user irrevocably selects m and satisfies the commitment. Consequently, subsequent locking scripts can enforce this selection via the $\text{CheckLampComm}_{pk_m}(m, c_m)$ spending condition.

Furthermore, we employ the *connector outputs* technique to enforce mutual exclusivity within a set of transactions. This mechanism leverages Bitcoin’s SIGHASH flags primitive to bind multiple candidate transactions to a single, shared unspent transaction output. Once one transaction consumes this connector output, the remaining candidates become invalid double-spends. We provide a more complete description of SIGHASH flags and connector outputs in Appendix B.2.1 and Appendix B.2.2 respectively.

Example. We illustrate the notation we use in the rest of this work with the following example: Let tx be a transaction that consumes outputs from two previous transactions, $tx1$ and an arbitrary transaction $*$, and creates two new outputs: Here, the transaction tx spends (i) the first output of $tx1$ which

tx	
<i>Inputs</i>	(0) ($tx1, 0, \text{CheckSig}_{pk_A} \wedge \text{RelTimelock}(10)$) (1) ($*$)
<i>Outputs</i>	(0) (4 BTC, $\text{AbsTimelock}(500)$) (1) (2 BTC, CheckSig_{pk_B})
<i>Witness</i>	(0) σ_A (1) $*$

requires a signature from user A and is locked under a relative timelock of 10 blocks, and (ii) an additional output denoted by $*$. We use $*$ to denote valid inputs and witnesses that carry coins needed for the protocol but have logic irrelevant to the protocol. The witness for the first input includes σ_A , authorizing the spend. The transaction creates two outputs: one holding 4 BTC, locked until block height 500, and another holding 2 BTC, spendable by user B .

2.3 Bitcoin Covenant Emulation

Although Bitcoin Script lacks native support for covenants, they can be emulated via pre-defined transaction trees. Consider the tree of all possible transaction sequences originating from the outputs of a root transaction Tx . A *Covenant Emulation System* (CES) for Tx is defined as a specific subtree rooted at Tx . By limiting valid execution paths to this subtree, a CES constrains how funds are spent not only for Tx , but recursively for all its descendants. We refer to the Bitcoin spending condition that enforces the rules of a CES as CheckCovenant .

In this work, we emulate covenants using a signer committee. In the “Signer Committee” Subsection of Section 3 we give an informal description of our covenant emulation protocol. We formalize the notion of CES and prove that our instantiation realizes it in Appendix A. Finally, we formally describe our covenant emulation protocol in Appendix E.1.

2.4 System and Threat Model

We consider a bridge protocol that enables the movement of bitcoins, denoted BTC, between the Bitcoin blockchain ($\mathcal{L}_A$) and a secondary ledger ($\mathcal{L}_B$), where wrapped coins are denoted wBTC. The protocol supports two primary operations:

Peg-in : A user (Alice) locks u units of BTC on $\mathcal{L}_A$. These are then minted as wrapped BTC (u wBTC) on $\mathcal{L}_B$.

Peg-out : A user (Bob) initiates a peg-out by burning u wBTC on $\mathcal{L}_B$ and claiming an equivalent amount of BTC on $\mathcal{L}_A$. The burned u wBTC originated from Alice’s peg-in, possibly through a sequence of transfers.

Besides the *users* of the bridge that perform the peg-in and peg-out (i.e., Alice and Bob), the bridge protocol involves:

Signer committee ($S_1, \dots, S_n$) : A set of n signers is responsible for setting up the bridge instance by signing necessary transactions and generating ephemeral keys used to authorize minting and define valid burn conditions for wBTC. For liveness, we assume that the committee is honest during setup for each bridge instance. For safety against dynamic adversaries, we assume that after setup at least *one signer behaves honestly and deletes the key*.

Operators ($O_1, \dots, O_m$) : Operators execute the bridge logic by monitoring both ledgers and facilitating peg-outs. Upon detecting a valid burn of wBTC on $\mathcal{L}_B$, an operator pays the user on $\mathcal{L}_A$ using its own funds, and is later reimbursed by claiming the pegged-in BTC. We assume that *at least one operator is honest* during execution; the rest may be Byzantine.

We assume that at least one honest operator has sufficient liquidity to front the user. We argue that this assumption is reasonable: since the required amount is known at the time of the peg-in, operators can anticipate their future liquidity needs and, if necessary, borrow funds to meet them. Moreover, operators with sufficient funds can reuse the same liquidity to serve multiple users sequentially, as they regain the funds each time.

Challengers : Challengers monitor $\mathcal{L}_A$ and ensure the security of peg-out by disputing invalid claims by (malicious) Operators. The role is *permissionless*; any party may act as a challenger, including signers, operators, and users. At least *one honest challenger is assumed to exist at all times*, while everyone else can be Byzantine.

Our construction is *trust-minimizing*: it relies only on *existential honesty* for signers, operators, and challengers. Notably, the challenger role is *fully permissionless*, allowing any party to participate in dispute resolution. This stands in contrast to prior bridge protocols, which typically require an *honest majority* assumption and often restrict the challenger role to a permissioned set. We further note that on platforms supporting covenants, both signers and operators become redundant, further reducing trust assumptions.

Cryptographic assumptions. All participants are computationally bounded. We rely on standard primitives: collision-resistant hashes, EUF-CMA secure signatures (e.g., Schnorr, Lamport), and succinct non-interactive arguments of knowledge (SNARKs) with perfect completeness and knowledge soundness. Full definitions are given in Appendix C.

2.5 BITVM2-BRIDGE Protocol Overview

BITVM2-BRIDGE implements a trust-minimizing bridge protocol that enables secure coin transfer between Bitcoin and

another ledger (e.g., a Layer 2 protocol) through an optimistic design. The protocol allows arbitrary off-chain computations to be validated on Bitcoin via fraud proofs. At its core is BITVM2-CORE, a mechanism that expresses and enforces computation using Bitcoin Script.

2.5.1 Strawman: a bridge with native covenants

To build intuition, consider a natural bridge design in a hypothetical version of Bitcoin with native covenant support. Alice wishes to bridge BTC to a sidesystem and locks BTC into a Bitcoin UTXO whose spending conditions fully encode the bridge logic. Alice transfers the corresponding asset on the sidesystem to Bob, who later wishes to peg out by burning the sidesystem coins. Bob can unlock the original Bitcoin UTXO by publishing a transaction, which we call Payout, that includes a proof that the sidesystem coins were burned. A covenant script ensures that the recipient of Payout is the same public key specified in the burn transaction.

This design cannot be implemented in Bitcoin today for two reasons: (a) Bitcoin does not support native covenants and therefore cannot restrict how the Payout transaction spends the locked coins. (b) Bitcoin Script cannot verify a proof of burn from an external sidesystem. To address the first limitation, we use *covenant emulation* via a signer committee and a *set of permissioned operators* that front funds to users. To address the second, we rely on an *on-chain dispute mechanism* that allows any party to force verification when a withdrawal is published. We describe this Bitcoin-compatible design next.

2.5.2 High-level BITVM2-BRIDGE flow

Peg-in. BITVM2-BRIDGE begins when a user, Alice, initiates a peg-in by creating (but not yet posting on-chain) a transaction, PegIn, that locks BTC on Bitcoin (ledger $\mathcal{L}_A$). Alice sends the peg-in request to the signer committee, which in response, generates ephemeral keys and collaborates with a set of designated operators to prepare *presigned* Bitcoin transactions. These transactions are designed to enforce correct behavior at peg-out. Looking ahead, the presigned transactions are Assert, Payout, PayoutOptimistic, Claim, Challenge (Section 3), and PegOut (Section 5). Once the setup completes, the signers send transactions to Alice. After Alice verifies that she has received all the necessary presigned transactions, ensuring that the PegOut procedure can be executed successfully when needed, she posts PegIn on $\mathcal{L}_A$. Subsequently, she is credited the equivalent amount of wrapped BTC (denoted wBTC) on an external ledger ($\mathcal{L}_B$). This step is done trustlessly with the use of a light client [7] or atomic swap protocol [17, 31].

Transfer and payment. Once minted, the wrapped BTC can be transferred across users on $\mathcal{L}_B$ as regular coins. These coins

may change hands arbitrarily,¹ and any user holding wBTC can later initiate a peg-out back to Bitcoin.

Peg-out. Suppose Bob becomes the owner of the wrapped BTC and decides to bridge the funds back to $\mathcal{L}_A$. To do this, Bob burns the wrapped BTC on $\mathcal{L}_B$ and requests a peg-out. An honest operator, observing the burn, transfers an equivalent amount of BTC to Bob by publishing PegOut on $\mathcal{L}_A$, using their own liquidity. This concludes Bob’s role in the protocol: he has been paid back on Bitcoin.

Operator reimbursement and dispute resolution. The operator must now reclaim the BTC they provided to Bob. This is where BITVM2-CORE comes into play: The operator publishes the presigned Claim transaction on $\mathcal{L}_A$, which triggers a dispute window. If no one challenges the Claim during the dispute window, the operator thereafter claims the PegIn coins by publishing the presigned PayoutOptimistic transaction. However, if the Claim is fraudulent, i.e., if Bob never burned the wrapped BTC or the operator failed to pay him, an honest challenger will submit a dispute (presigned Challenge transaction) to prevent the reimbursement.

To justify the claim, the operator may then publish a pre-signed Assert transaction, committing to an argument of the statement that (i) a valid burn occurred on $\mathcal{L}_B$ and (ii) a corresponding payment by the operator occurred on $\mathcal{L}_A$. The Assert triggers a second dispute period. If the commitment made in Assert is incorrect, an honest challenger publishes a Disprove transaction during the dispute period.

A valid Disprove will prevent the *malicious operators* from taking the PegIn coins. Since the PegIn output remains unspent, honest operators can later claim the PegIn coins as reimbursement after successfully fronting coins to a user.

If no successful Dispute is published on-chain before the deadline, the PegIn coins are awarded to the (*honest*) operator by publishing the presigned Payout transaction. Malicious challengers cannot construct a valid Disprove, and thereby the operator will reclaim the PegIn coins.

A high-level illustration of the BITVM2-CORE protocol architecture and challenge flow is shown in Fig. 2.

2.5.3 Intuition on BITVM2-CORE and light client

Implementing Assert in Bitcoin Script. At the heart of our construction lies an interactive protocol, termed BITVM2-CORE, which is realized using Bitcoin Script. BITVM2-CORE enables honest operators to reclaim their funds (via Assert), while preventing malicious operators from withdrawing funds from the bridge (via Disprove). *But how can Assert express such a complex statement on Bitcoin Script?*

The key idea is to use Assert to commit to the output of a SNARK verifier attesting that the burn and payment indeed occurred. The core technical challenge lies in expressing the

SNARK verifier logic on Bitcoin Script, which is inherently constrained. To overcome this, we observe that Bitcoin Script can express arbitrary total functions with statically bounded control flow,² that is, deterministic computations that are guaranteed to terminate and whose execution bounds are fixed at script construction time.

Based on this observation, we implement the verifier logic as a SNARK verifier circuit and compile it to Bitcoin Script. Since a full SNARK verifier typically exceeds Bitcoin’s block size,³ we decompose it into sequential chunks, each small enough to fit in a Bitcoin transaction (max 4MB). The Assert transaction commits to each chunk’s code and output. During a successful dispute, a Disprove transaction reveals the true output of a specific chunk by re-executing the chunk on-chain and comparing it against the operator’s claim. This enables a full on-chain fraud proof mechanism without any changes to Bitcoin. To ensure that Assert encodes the desired program, it is presigned by the signer committee for each operator during setup. Additionally, we assume the program is public knowledge, e.g., it may be available in a public repository on the Internet. This ensures its availability to challengers and any other participants who can independently verify it. The precise design of Assert is iteratively refined in strawman designs S1–S5 and finalized in the Optimistic BITVM2-CORE description in Section 3.

Verifying transaction inclusion. Currently, BITVM2-CORE must verify two key events: the inclusion of the peg-out transaction on Bitcoin (where the operator pays Bob), and the inclusion of Bob’s burn transaction on $\mathcal{L}_B$. We assume that $\mathcal{L}_A$ (Bitcoin) includes state commitments of $\mathcal{L}_B$ and discuss alternative designs in Section 8. This reduces the task of verifying transaction inclusion on $\mathcal{L}_B$ to verifying a Bitcoin transaction against a known state commitment; thereby enabling both verifications to be handled uniformly within Bitcoin. To realize this, we present in Section 4, to our knowledge, the first *on-chain light client for Bitcoin*, built using BITVM2-CORE.

2.6 Protocol Goals

In the rest of this section we introduce our primitive in stages: we begin by exposing informally the BITVM2-CORE protocol goals, then outline the requirements for the light client protocol, and finally define the desiderata for the full BITVM2-BRIDGE system. We defer formal definitions to Appendix D.3.

BITVM2-CORE. BITVM2-CORE is a protocol run between a prover P (“operator” in BITVM2-BRIDGE) and a set of verifiers V (“challengers” in BITVM2-BRIDGE), who take part in a distributed ledger protocol $\Pi_{\mathcal{L}}$ as full nodes.

BITVM2-CORE enables P to prove on-chain the output of a known function f . The prover is entitled to publish a

¹Before accepting the coin on $\mathcal{L}_B$, the recipient should check that $\mathcal{L}_A$ contains a corresponding PegIn transaction.

²This can be done via opcodes such as OP_ADD and OP_IF.

³For example, 900MB for Groth16.

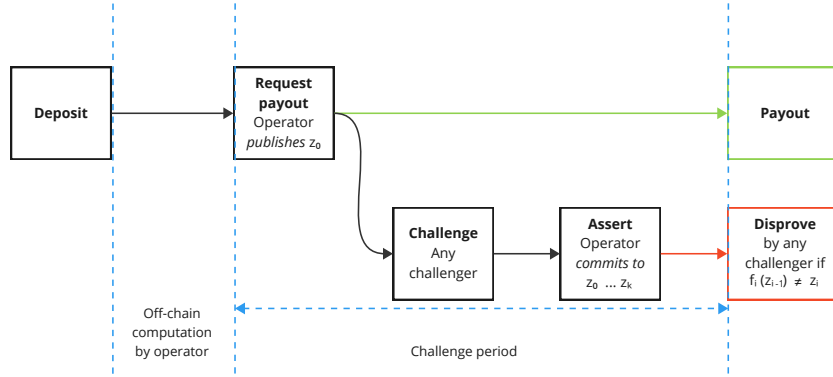


Figure 2: Simplified overview of the BITVM2-CORE transaction flow. The SNARK verifier program $f(x) = y$ is split into $f_1, f_2, \dots, f_k$ with intermediary results $z_0, \dots, z_k$. A challenged operator must reveal the intermediary states in the Assert transaction. This allows a challenger to disprove a false claim of an operator by executing the disputed sub-program f_i in a Bitcoin transaction.

specific transaction conditioned on successful verification of the computation’s result. For example, P may lock a deposit to qualify as a service provider for the computation of f in exchange for a service fee. In such a case, P can only reclaim their funds upon V verifying the result of the computation f .

At a high level, BITVM2-CORE satisfies two key properties: (i) **completeness**, i.e., an honest prover that knows the input of the correct execution (i.e., a valid witness) can always publish a prespecified transaction on-chain (e.g., the reclaim of the deposit), and (ii) **soundness**, that is, a malicious prover without a valid witness cannot publish such transaction.

Light client. The security definition of the light client protocol is based on BITVM2-CORE. Intuitively, the light client is secure if upon termination, it outputs a block that: (i) is guaranteed to appear in the future chains of all honest parties (**safety**), and (ii) is recent enough, i.e., reflects the honest network’s progress (**liveness**). We term this block as **admissible**.

BITVM2-BRIDGE. At a high level, a *bridge* connects two ledgers, say $\mathcal{L}_A$ and $\mathcal{L}_B$, and ensures that actions on one ledger (the *source*) trigger corresponding actions on the other (the *destination*). For example, locking coins on $\mathcal{L}_A$ should lead to minting coins on $\mathcal{L}_B$. To keep track of such cross-chain transfers, we associate each with a unique identifier id , which links all related transactions across both ledgers.

A secure bridge must satisfy two fundamental properties:

- **Bridge liveness.** If a valid transaction (e.g., deposit or other action) appears on the source ledger $\mathcal{L}_A$, then within bounded time the corresponding transaction (identified by id) will appear on the destination ledger $\mathcal{L}_B$.
- **Bridge safety.** If a transaction appears on the destination ledger $\mathcal{L}_B$, then the corresponding transaction (identified

by id) must appear on the source ledger $\mathcal{L}_A$ (possibly observed slightly earlier or within a bounded delay).

Bridge liveness ensures that honest users who lock coins on $\mathcal{L}_A$ can be confident that they will eventually receive the corresponding coins on $\mathcal{L}_B$. Bridge safety on the other hand prevents “printing coins out of thin air”, ensuring that the total supply across the two systems is preserved. If both properties hold, we call the bridge *robust*.

Finally, we distinguish between a *unidirectional bridge*, where assets move only one way (e.g., from $\mathcal{L}_A$ to $\mathcal{L}_B$), and a *wrapped-asset bridge*, which combines two unidirectional bridges to enable bidirectional transfers. In the latter, a *peg-in* procedure locks coins on $\mathcal{L}_A$ and mints wrapped coins on $\mathcal{L}_B$, while a *peg-out* procedure burns wrapped coins on $\mathcal{L}_B$ and unlocks the original coins on $\mathcal{L}_A$. Bidirectional robustness ensures that assets can safely circulate between ledgers.

Bridge operator safety. In our design, peg-outs are executed by operators who advance their own liquidity to clients. Hence, peg-out liveness in practice requires that honest operators can always recover their funds, even against Byzantine adversaries. We refer to this guarantee as *Bridge Operator Safety*. It refines peg-out liveness by ensuring honest operators face no risk of loss and thus remain incentivized to service exits; an essential condition for deployability in practice.

3 BITVM2-CORE: General Function Verification on Bitcoin

Bitcoin’s scripting language is intentionally limited in expressiveness, lacking support for loops, recursion, or general-purpose computation. Nevertheless, any total function can be encoded as a Bitcoin Script program. Yet, in practice the 4MB

maximum block size and the 1000 element stack limit⁴ prevent encoding large programs, like SNARK verifiers, directly on-chain.

In this section, we present BITVM2-CORE, a general function verification mechanism that enables any total function to be executed off-chain and verified on Bitcoin through a dispute resolution protocol. This mechanism forms the foundation of our bridge protocol but also stands as a general-purpose construction of independent interest.

The goal of BITVM2-CORE is to allow an operator O to assert a statement of the following form: that a Bitcoin Script program g executed on input x returns $y = g(x)$. The operator stakes a deposit of d BTC, where $d \geq 0$, and posts a commitment to this claim on-chain. If the claim is correct, they reclaim the deposit. Otherwise, any party can challenge the claim by executing a portion of the program on-chain and disproving the claim, preventing the operator from reclaiming their deposit.

We illustrate the design rationale through progressively refined strawman constructions. As a running example, we consider a complex function g that takes as input a blockchain C_Z from genesis till a desired height h and returns whether C_Z is valid – verifying expensive cryptographic procedures such as transaction validity and consensus rules.

(S1) Direct function verification

In our initial strawman, during setup, O posts an Assert transaction that consumes the output of a deposit and encodes in Bitcoin Script the entire program g . We stress that at setup O may not yet know a pair x, y such that $g(x) = y$, e.g., C_Z is not yet of height h .

When O learns x, y (e.g., C_Z reaches height h), they publish a Payout transaction that spends Assert by providing x, y and the signature of the operator σ_O . The equality $y = g(x)$ is verified directly on-chain and the deposit coins are moved to the output of Payout. If $y \neq g(x)$, the Payout transaction is invalid, hence O cannot reclaim their deposit.

While conceptually straightforward, this approach encounters a fundamental issue: both the script g (e.g., blockchain validation rules) and the input x (e.g., a chain C_Z) far exceed Bitcoin’s block size and stack limits by orders of magnitude. As a result, neither the Assert nor the Payout transactions can be posted on-chain, as both the output script of Assert, g , and the witness of Payout, x , exceed Bitcoin’s limits.

(S2) Function verification in a SNARK

We first address the large witness size of Payout by compiling g into a SNARK. The output of Assert now encodes a SNARK verifier for g , making the witness z provided in Payout succinct. In our example, the operator can spend Payout by publishing a succinct proof attesting to the existence of a chain

C_Z of height h . However, the previous challenge persists: *the SNARK verifier itself is too large* to fit within Bitcoin’s block size constraints, and thus cannot be embedded in Assert.

(S3) Splitting the computation

To overcome Bitcoin’s block size limitations, we split the SNARK verifier f into a sequence of smaller sub-programs $f_1, \dots, f_k$, each of which *can* be executed within a Bitcoin transaction. Our goal is to design a mechanism such that, in the event of operator misbehavior, only a single sub-program must be verified on-chain. To achieve this, we introduce a fraud-proof-based protocol with the following objectives: (i) ensure that an honest operator is able to reclaim their deposit if they compute and provide all intermediate results $z_1, \dots, z_k$ such that each sub-computation $z_i = f_i(z_{i-1})$ is correct for the input $z_0 = x$, and (ii) allow any party to challenge and disprove a faulty intermediate result z_i if the operator is malicious.

To achieve this, the operator must provide all intermediate states of the computation $z_0, z_1, \dots, z_k$, where $z_k = f(z_0)$, when posting the Assert. Without this, a challenger has no basis for disproving the computation, as they do not know the intermediate states z_i . We therefore modify the deposit and Assert transactions as follows: The locking script of the deposit’s output requires the z_i in the witness of Assert. The output of Assert uses a Taproot structure with $k + 1$ Tapleaves:

Happy path (Payout). The first leaf allows the operator to reclaim their deposit after a predefined delay Δ_A by publishing a Payout transaction containing their signature. This path can only be used if no challenge is submitted during the delay.

Unhappy path (Disprove). The remaining k leaves allow anyone to challenge the correctness of a sub-program f_i . Each Tapleaf contains a script that checks whether the claimed output z_i was correctly derived from input z_{i-1} via f_i . To dispute the claim, a challenger publishes a Disprove transaction with the relevant values and demonstrates that $z_i \neq f_i(z_{i-1})$, proving the operator lied.

If a valid challenge is submitted within the window Δ_A , the operator loses the deposit. An overview of the resulting protocol is shown in Fig. 3.

However, now Assert can only be published at runtime, as it requires the intermediate results $z_1, \dots, z_k$ of the computation which are not known at setup. As a result, *the operator is not bound to the function f during setup*, allowing a malicious operator to cheat and reclaim the deposit by using a different function f .

(S4) Signer committee

To enforce that f is fixed at setup, we employ a signer committee of n signers $S_1, \dots, S_n$. The signers are assumed to be

⁴<https://tinyurl.com/mt7ex7xh>

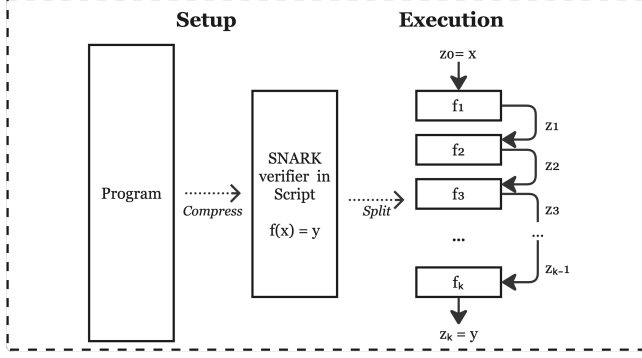


Figure 3: A SNARK verifier, representing an arbitrary program, is compiled to Bitcoin Script and split into sub-programs $f_1, \dots, f_k$, each small enough to run in a block.

honest during setup to ensure it completes, but they cannot steal the operator's funds.

During setup, the operator O contacts the signer committee to authorize the validity of the Assert transaction, thereby binding themselves to the execution of f : Each signer generates a fresh key pair and uses it to presign both Assert and Payout. The signers then delete their keys. The resulting UTXOs are locked with an n -of- n multi-signature condition CheckMultiSig_C as well as the signature of the operator. We refer to the signer committee mechanism as CheckCovenant .

As long as at least one signer is honest and deletes their key after signing, O cannot deviate from the intended protocol. Once Assert is posted, O can only reclaim their deposit by following the prescribed execution and providing correct results. We stress that, unlike similar constructions relying on honest majority committees, our approach requires only *existential honesty* among signers – a much weaker assumption.

This construction ensures incorrect claims can always be challenged. However, it *does not prevent dishonest challengers from falsely disputing honest executions*, since the script does not enforce that the z_i^c used to challenge (in Disprove) match the z_i that the operator published in Assert. We address this next.

(S5) Lamport signatures

To prevent malicious challengers from forging arbitrary intermediate states in a Disprove transaction, we require the operator O to publish both the intermediate states z_i and their signatures σ_i in the Assert transaction. Each Disprove must include z_{i-1}, z_i and their valid signatures σ_{i-1}, σ_i in its witness. While Bitcoin does not natively support verification of signatures over arbitrary messages, we can manually implement verification of Lamport signatures in Bitcoin Script using the native opcodes for hashing. We now present the full protocol in four phases, illustrated in Fig. 4:

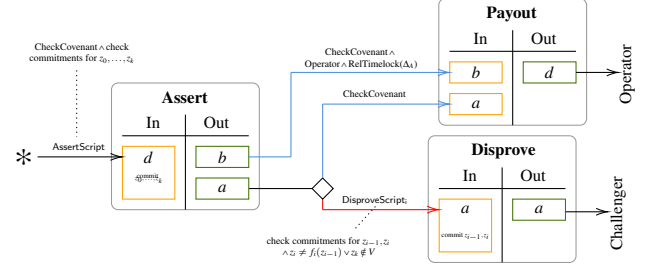


Figure 4: Illustration of the transactions Assert, Disprove, and Payout. The input of Assert can be any UTXO (*), but its spending script must be a Covenant including commitments to $z_0, \dots, z_k$. For readability, we use: gray rounded rectangles for transactions, green for BTC in outputs, and orange for BTC spent. Blue arrows indicate operator actions, red arrows indicate other participants. Arrow labels show spending conditions. Gray dashed boxes highlight the parts of transactions hashed and presigned under non-SIGHASH_ALL flags.

Phase 1: setup. For each z_i , the operator O generates a Lamport keypair (sk_{z_i}, pk_{z_i}) and embeds the public keys in the Assert and Payout transactions. These transactions are then sent to the signer committee, which presigns them to enforce the desired execution. The presigned transactions are returned to O and can be posted later. This setup phase is fully off-chain and is formalized in Section 3 and illustrated in Fig. 4.

Phase 2: off-chain execution. The operator O executes the sub-programs $f_1, \dots, f_k$ off-chain with input $z = z_0$ and produces the intermediary states $z_1, \dots, z_{k-1}$ as well as the final state z_k . This output z_k should be $\{\text{True}\}$.

Phase 3: commit and challenge. The operator publishes the Assert transaction on-chain, claiming to have correctly executed the program by providing the full trace $z_0, \dots, z_k$. This transaction spends the deposit's output (indicated by *) holding d BTC, which must satisfy two conditions: (i) it must be spendable only via the Assert transaction (via connector outputs), (ii) it must provide valid Lamport signatures $c_{z_0}, \dots, c_{z_k}$ verifiable against the public keys $pk_{z_0}, \dots, pk_{z_k}$.

Assert Transaction	
Inputs	(0) $(*, *, \text{AssertScript} := \text{CheckCovenant} \wedge \text{CheckLampComm}_{pk_{z_0}} \wedge \dots \wedge \text{CheckLampComm}_{pk_{z_k}})$
Outputs	(0) $(b \text{ BTC}, \langle \text{RelTimeLock}(\Delta_A) \wedge \text{CheckCovenant} \wedge \text{CheckSig}_{pk_O} \rangle)$ (1) $(a \text{ BTC}, \langle \text{CheckCovenant}, \text{DisproveScript}_1, \dots, \text{DisproveScript}_k \rangle)$
Witness	(0) $*, \text{Covenant}, z_0, \dots, z_k, c_{z_0}, \dots, c_{z_k}$

Note that a denotes a small payout for the challenger that successfully disproves Assert, while $b = d - a$. More details

follow in Phase 4.

Phase 4: resolution. If no one publishes a Disprove transaction within the challenge period Δ_A , the operator can redeem their deposit by publishing the Payout:

Payout Transaction	
<i>Inputs</i>	(0) (Assert, 0, $\langle \text{RelTimelock}(\Delta_A) \wedge \text{CheckCovenant} \wedge \text{CheckSig}_{pk_o} \rangle$) (1) (Assert, 1, $\langle \text{CheckCovenant} \rangle$)
<i>Outputs</i>	(0) (d BTC, CheckSig_{pk_o})
<i>Witness</i>	(0) Covenant, σ_O (1) Covenant

To dispute a false claim (i.e., some $z_i \neq f_i(z_{i-1})$), any observer can submit a Disprove transaction, which spends the second output of Assert. It satisfies one of the last k Tapleaf scripts, which are tied to sub-programs f_i . These scripts verify that the committed intermediate state is incorrect using the public commitments from Assert.

Disprove Transaction	
<i>Inputs</i>	(0) (Assert, 1, $\langle \text{DisproveScript}_i \rangle$)
<i>Outputs</i>	(0) (a BTC, True)
<i>Witness</i>	(0) Covenant, $z_{i-1}, z_i, c_{z_{i-1}}, c_{z_i}$

The output determines the fate of the operator's deposit d BTC: a portion b BTC is burned (i.e., cannot be reclaimed), and the remainder $a = d - b$ may be claimed by the challenger. We partially burn the operator's deposit d BTC to ensure that any unsuccessful claim incurs a real penalty. Without burning part of the deposit, a malicious operator could submit PayoutOptimistic claims for free: if challenged, they could simply disprove their own false claim and recover both the challenge fees and their full deposit⁵. For simplicity, we denote this in the above definition as (a BTC, True). We provide the script logic in Algorithm 1.

Optimistic BITVM2-CORE

To further reduce on-chain costs, we introduce an optimistic mode where the operator initially posts a lightweight Claim, asserting knowledge of a valid input-output pair (z, o) for some $f(z) = o$, without revealing the full computation. If no challenger disputes the claim by posting a Challenge transaction within delay Δ_B , the operator withdraws via PayoutOptimistic, bypassing full verification.

⁵This mechanism is enforced using the SIGHASH_SINGLE flag [32], which commits to specific input-output pairs, while allowing more outputs to be added without a signature (e.g., bounty payouts, cover transaction fees).

Algorithm 1 The DisproveScript_i for each Tapleaf $i \in \{1, \dots, k\}$. The algorithm takes as input the committee's multisignature σ_C , the intermediate states z_{i-1}, z_i of the program f , and the Lamport commitments $c_{z_{i-1}}, c_{z_i}$.

```

1: function DisproveScripti( $\sigma_C, z_i, z_{i-1}, c_{z_i}, c_{z_{i-1}}$ )
2:   CheckCovenant()
3:   CheckLampComm $pk_{z_{i-1}}$ ( $z_{i-1}, c_{z_{i-1}}$ ) Both  $pk_{z_{i-1}}$  and  $pk_{z_i}$  are hard-coded in the script
4:   CheckLampComm $pk_{z_i}$ ( $z_i, c_{z_i}$ )
5:   return  $z_i \neq f_i(z_{i-1}) \vee (i = k \wedge z_k \notin V)$   $f_i, V$  and  $k$  are hard-coded in the script
6: end function

```

If challenged, the operator must fall back to the previous protocol, publishing Assert, allowing on-chain dispute resolution. Thus, optimistic verification substantially lowers on-chain footprint in honest scenarios while preserving security. The complete optimistic verification workflow is shown in Fig. 5. Here, we only highlight the differences to (S5).

Phase 1: setup. The signers presign PayoutOptimistic, enforcing the optimistic execution path.

Phase 3: commit and challenge. To initiate verification, the operator posts a presigned Claim transaction, asserting they know a pair z, o such that $f(z) = o$. They can reveal z either on-chain or off-chain (e.g., via a data availability layer) to enable external verification. With AssertScript defined in (S5), let Claim:

Claim Transaction	
<i>Inputs</i>	(0) (*)
<i>Outputs</i>	(0) (d BTC, $\langle \text{RelTimelock}(\Delta_B) \wedge \text{CheckCovenant}, \text{AssertScript} \rangle$) (1) (0 BTC, CheckSig_{pk_o})
<i>Witness</i>	(0) $\sigma_O \wedge *$ (1) z

During the challenge period Δ_B , any party may dispute Claim by posting a presigned Challenge transaction, forcing the operator to respond with a corresponding Assert transaction, which can then be contested via a Disprove. The second output of Claim serves as a *connector output* (for the definition of connector outputs, see Appendix B.2.2); it must remain unspent for the operator to use PayoutOptimistic and reclaim their deposit unchallenged.

Phase 4: resolution. If unchallenged after Δ_B , the operator posts PayoutOptimistic to claim the funds and disable the dispute logic.

We prove BITVM2-CORE satisfies both completeness and soundness (Definitions D.4 and D.5) assuming existential honesty of signers, operators, and challengers in Appendix F.

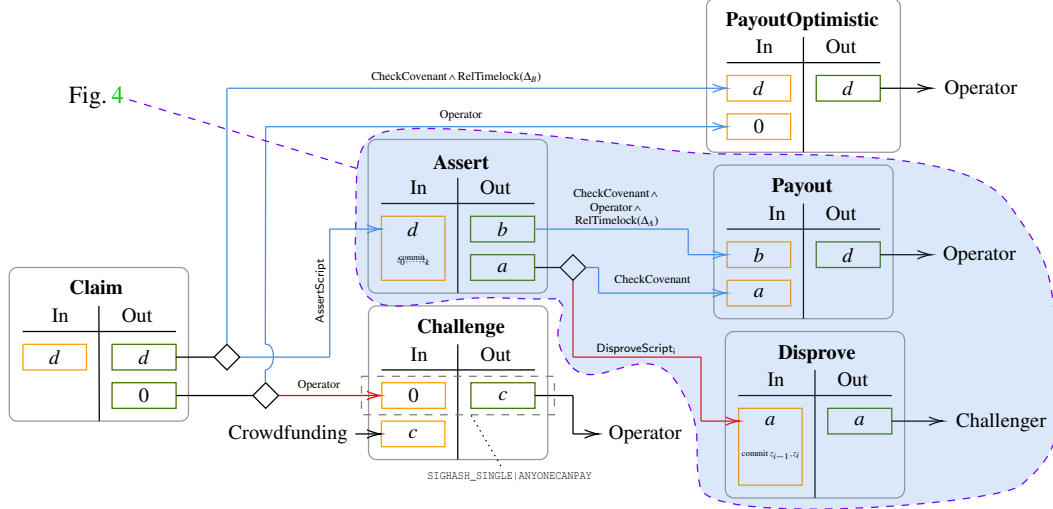


Figure 5: Illustration of the optimistic function verifier of BITVM2-CORE.

Challenge Transaction	
Inputs	(0) (Claim, 1, CheckSig _{pk_O}) (1) (*)
Outputs	(0) (c BTC, CheckSig _{pk_O})
Witness	(0) σ_O (1) *
PayoutOptimistic Transaction	
Inputs	(0) (Claim, 0, $\langle \text{RelTimelock}(\Delta_B) \wedge \text{CheckCovenant} \rangle$) (1) (Claim, 1, CheckSig _{pk_O})
Outputs	(0) (d BTC, CheckSig _{pk_O})
Witness	(0) Covenant (1) σ_O

Reward and collateral. To deter a malicious challenger from grieving an honest operator by repeatedly triggering costly Assert transactions, the protocol requires the challenger to post collateral c BTC that covers the operator's on-chain costs. This collateral can be crowdfunded, allowing multiple users to jointly contribute inputs to the Challenge transaction. See Section 8 for details.

4 On-chain Bitcoin Light Client

We now build an on-chain Bitcoin light client for static difficulty using BITVM2-CORE. The goal is to let Bitcoin verify that a given block belongs to the canonical proof-of-work

chain (i.e., stable chain adopted by honest nodes) and is sufficiently recent. Note that this is a light client of Bitcoin on the Bitcoin chain, i.e., it verifies to Bitcoin the presence of a block in its consensus. As such, it provides an *introspection capability* to Bitcoin, something that it does not have natively.

To this end, we instantiate BITVM2-CORE's AssertScript and DisproveScripts with a statement that checks whether a candidate block is part of a valid PoW chain from genesis to the declared tip (cf. Algorithm 2). At a high level, the operator commits to the chain tip and provides a succinct proof attesting to its validity, which is embedded in the Assert transaction. More concretely, the operator signs a sequence of intermediate states $z_0, \dots, z_k$ with Lamport keys. The first state z_0 encodes the chain tip, its height, and a SNARK proof that this tip extends a valid proof-of-work chain from genesis. From this commitment, the admissible block is set to the block $m+k$ positions before the tip, ensuring that it is both recent and stable (i.e., the light client protocols is live and safe, respectively).

Assert Transaction	
Inputs	(0) (Claim, 0, AssertScript)
Outputs	(0) (b BTC, CheckCovenant) (1) (a BTC, $\langle \text{CheckCovenant} \wedge \text{RelTimelock}(\Delta), \text{DisproveScript}_1, \dots, \text{DisproveScript}_k, \text{CheckLampComm}_{pk_h} \wedge \text{AbsTimelock}(h + 3\Delta) \rangle$)
Witness	(0) *, Covenant, $z_0, \dots, z_k, c_{z_0}, \dots, c_{z_k}$

The timelock AbsTimelock($h + 3\Delta$) ensures freshness: if an operator tries to commit to an outdated block, challengers can disprove it before payout. The only change from the stan-

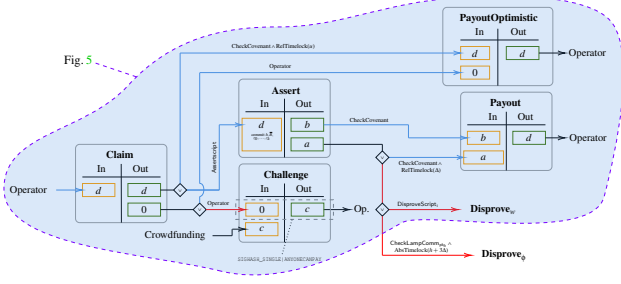


Figure 6: Illustration of the on-chain Bitcoin light client. Its goal is to verify to Bitcoin that a block B is in the ledger of Bitcoin. It does so by verifying that 1) the B is in a valid PoW chain C , 2) the chain C is a prefix of the stable chain adopted by honest Bitcoin miners. The former is achieved using BITVM2-CORE (Fig. 5) and the latter is achieved by verifying that the height h of C is close to the length of the longest Bitcoin chain. This latter verification is piggy-backed onto BITVM2-CORE by adding a Lamport signed height h in Assert, and adding a Disprove path to allow challenge if the height is not close to the height of the longest chain.

dard protocol is this added spending condition (see Fig. 6). Setting the timelock to 3Δ (Δ being the block confirmation window) gives an honest operator sufficient time to post Assert, wait for confirmation, and publish Payout.

Intuitively, this construction guarantees that the admissible block is both *live*, since it is chosen close to the tip of the chain (within $m+k$ blocks), and *safe*, since an adversary cannot extend a competing chain fast enough to replace it. We formalize this in Theorem 6.3 and prove it in Appendix G.

Algorithm 2 The chainstate proof Π_{lc} is the statement checked by the SNARK. It consists of (ϕ, w) , where $\phi = (\bar{B}, h)$ is a block and its height, and w is a chain of blocks of length $|w| > m+k$. The algorithm verifies that w is a valid chain (each block B has a correct hashpointer $B.parent$, and a hash $\mathcal{H}(B)$ below the PoW-target T) that connects the (hard-coded) genesis block B_0 with $\bar{B}$ at height h .

```

1: function  $\Pi_{lc}(\phi, w)$ 
2:   for  $i \in \{0, \dots, |w| - 2\}$  do
3:     if  $w[i+1].parent \neq \mathcal{H}(w[i]) \vee \mathcal{H}(w[i+1]) \geq T$ 
       then
4:       return False
5:     end if
6:   end for
7:   return  $B_0 = \mathcal{H}(w[0]) \wedge \phi.\bar{B} = \mathcal{H}(w[-1]) \wedge \phi.h = |w| - 1$ 
        $w[-m-k]$  is admissible
8: end function

```

5 The BITVM2-BRIDGE Design

A core application of BITVM2-CORE is enabling trust-minimized bridges between Bitcoin ($\mathcal{L}_A$) and other blockchains or Layer 2 networks ($\mathcal{L}_B$), collectively called *sidesystems*. As outlined earlier, the goal of our bridge is to securely represent Bitcoin on the sidesystem as a wrapped asset (wBTC), redeemable at a 1:1 ratio for BTC.

Bridges typically require trust assumptions, with current Bitcoin bridges relying heavily on multisignatures and honest majority assumptions [35]. While ideally, peg-in/out transactions could rely on native blockchain light clients, Bitcoin’s limited scripting capability and lack of covenants restrict direct implementation: since the receiver (Bob) of the peg-out is not known at the time of the peg-in, it is impossible to enforce that only Bob can unlock the v BTC.

Our design addresses this limitation by introducing a known operator set that fronts funds during peg-out, and then reclaims their funds by leveraging the Bitcoin light client introduced in Section 4. This setup fixes operators at setup, bypassing the unknown recipient issue.

For simplicity, we assume the sidesystem operates as a rollup, using Bitcoin as its underlying consensus layer. Specifically, the rollup state transitions are verifiable via data commitments published on Bitcoin. Formally, we define a modified statement for the SNARK within the Bitcoin light client, illustrated in Algorithm 3. This modification ensures the PegOut transaction is indeed reflected in the sidesystem state.

Algorithm 3 The chainstate proof Π , which defines the statement over which we define our SNARKs. It uses Algorithm 2 with input (ϕ, w) , to verify that w is a valid PoW chain. In addition, it checks (1) that a peg-out is embedded in a Burn on the sidesystem, and (2) that this Burn appears in the sidesystem state omitted in w SidesystemState(w) up to the admissible block.

```

1: function  $\Pi(\phi, w)$ 
2:   if PegOut  $\notin w[-m-k] \vee$  PegOut  $\notin$  Burn  $\vee$  Burn  $\notin$ 
       SidesystemState( $w[-m-k]$ ) then
3:     return False
4:   end if
5:   return  $\Pi_{lc}(\phi, w)$  cf. Algorithm 2
6: end function

```

Peg-in. Alice sends a peg-in request message to the signing committee, whose members create ephemeral keys and reply with the corresponding public key. With these keys, she constructs (but does not broadcast) the transaction PegIn, locking BTC on $\mathcal{L}_A$.

Alice shares PegIn with all operators who execute the setup of the Bitcoin light client (cf. Section 4), including PegIn as an additional input. Specifically, both the Payout and PayoutOptimistic transactions now require an extra input, spending the output of PegIn. This modified transaction graph is depicted in Fig. 7. Once operators complete this

BITVM2-CORE. First, we show that BITVM2-CORE satisfies *soundness*, i.e., no malicious operator without a valid witness can reclaim the deposit, and *completeness*, i.e., an honest operator can always reclaim their deposit.

Theorem 6.1 (Soundness). *If the SNARK has knowledge soundness, the ledger is safe and live, and a set of signers and a set of verifiers with existential honesty, BITVM2-CORE satisfies soundness.*

Theorem 6.2 (Completeness). *Assuming the SNARK is complete, the ledger is safe and live, and a set of signers with existential honesty, BITVM2-CORE satisfies completeness.*

Intuition. Soundness holds because an invalid proof would either require breaking SNARK soundness or a violation of ledger safety/liveness. Completeness holds because if the witness is valid, the payout must appear unless the SNARK is not complete or the ledger itself fails.

Bitcoin light client. We next analyze the light client of Section 4.

Theorem 6.3 (Informal Light Client Security). *The light client accepts only admissible blocks: with overwhelming probability, the accepted block is both recent (within $m+k$ blocks of the tip) and will remain in all honest chains.*

Intuition. Liveness follows because the chosen block is always close to the tip; safety follows from the Bitcoin’s immutability (common-prefix property [13]), which prevents an adversary from replacing such a block.

BITVM2-BRIDGE. Finally, we establish bridge security for both directions of transfer and for operator incentives.

Theorem 6.4 (Peg-in Safety). *The peg-in bridge is safe under the assumption that at least one signer is honest during setup and that Π_{L_A} satisfies ledger safety.*

Intuition. A forged mint would require all signers to collude or a valid peg-in to be removed from Bitcoin; both violate our assumptions.

Theorem 6.5 (Peg-in Liveness). *The peg-in bridge is live assuming (i) at least one honest operator exists during setup, (ii) all the signers are honest during setup, and (iii) both Π_{L_A} and Π_{L_B} are live.*

Intuition. An honest operator triggers the mint, honest signers authorize it, and ledger liveness ensures eventual inclusion on the sidesystem.

Theorem 6.6 (Informal Peg-out Safety). *The peg-out bridge satisfies safety assuming at least one honest signer post-setup, and L_B satisfies safety.*

Intuition. A peg-out on Bitcoin is possible only if a corresponding Burn exists on the sidesystem: all users are honest during setup, an honest post-setup signer additionally prevents fraudulent presigning, and sidesystem safety ensures valid Burns are not removed.

Theorem 6.7 (Informal Peg-out Liveness). *The peg-out bridge satisfies liveness, assuming all signers are honest during setup while at least one remains honest post-setup, L_A is safe and live, and an honest operator monitors L_B .*

Intuition. Once a valid Burn appears on the sidesystem, an honest operator will complete the peg-out, presigned transactions (from setup with at least one honest signer) allow redemption, and Bitcoin liveness ensures inclusion.

Assuming an honest challenger, BITVM2-BRIDGE also satisfies Bridge Operator Safety.

Theorem 6.8 (Informal Bridge Operator Safety). *An honest operator who performs a valid peg-out is guaranteed to recover their funds, assuming the signer committee is honest during setup and one member remains honest after, an honest challenger exists, both ledgers are live and safe, the light client is secure, and BITVM2-CORE is complete and sound.*

Intuition. An honest operator can fail to recover funds only if (i) another operator wrongfully claims the payout, or (ii) no payout appears despite a valid witness. The former cannot happen because BITVM2-CORE is sound, the light client is safe, and the signers are honest during setup and at least one signer is honest after (deleting their key). The latter cannot occur because the signers are honest during setup, the light client is live, and BITVM2-CORE is complete.

7 Evaluation

We provide a production-level reference implementation, which is the result of a large open-source effort, of which we are among the core contributors. The code repository of our [BitVM implementation](#) can be found here [1]. Additionally, we successfully submitted the BitVM transactions on the Bitcoin *mainnet*.

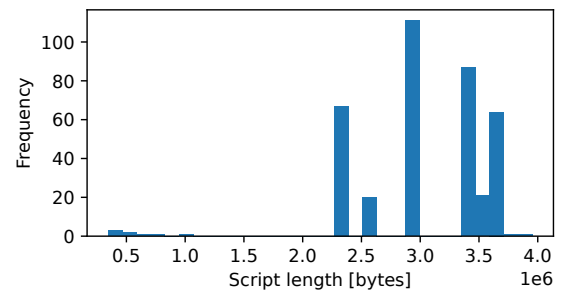


Figure 8: Histogram of script lengths in our implementation

The 379 Disprove scripts span roughly 500 kB to 3.99 MB, with dense peaks near ≈ 3 MB (pre-compute/hash phases), and a smaller tail above 3.5 MB for the largest composite steps. Most scripts therefore sit in the 2–3 MB band, with only a handful below half a megabyte or beyond 3.5 MB.

Based on historical fee data, the worst-case fee required to ensure inclusion of a transaction on Bitcoin mainnet within a two-week window has rarely exceeded ~ 50 sat/vB. At that price, a ~ 4 MB Disprove entails roughly 2 BTC of collateral per operator. Should fees spike higher, the transaction can still be confirmed through cooperative child-pays-for-parent (CPFP) bumping.

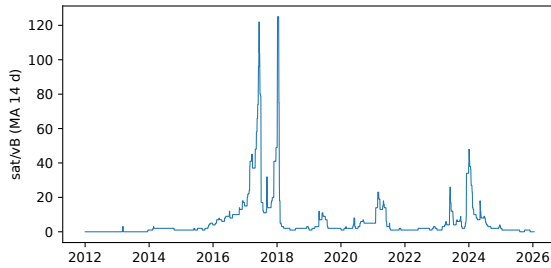


Figure 9: Fee-rate that gives a $\geq 95\%$ chance of confirmation within 14 days (mempool.space data).

On June 3 2025 we validated the full unhappy path on Bitcoin mainnet: starting with the Claim [4], followed by the Challenge [3], then the Assert [2], and finally the (largest) Disprove [5] transaction. The sequence confirmed in 42 blocks (7h 36min), demonstrating that a challenge period is viable even under mainnet congestion. The total cost was 14.9M sat ($\approx \$16,000$ as of June 2025), with Assert and Disprove accounting for 3.89M and 10.99M sat, respectively. This path is unlikely in practice, as honest operators avoid losing collateral and challengers must pay these fees in case their challenge fails (cf. Section 3). Rather, this amount reflects the collateral required per peg-in in BITVM2-BRIDGE. Reducing the size of the Assert and Disprove transactions remains an active area of research.

The happy path, which is the one normally executed, involves instead only PegOut, Claim, and PayoutOptimistic that remain below 16.75 vB, i.e., around 50k sat (roughly \$ 53). We discuss how to further reduce this cost to 534 vbytes, or around 1.6k sat (roughly \$ 1.66) in Section 8.

In practice, users will not directly interact with BitVM deposits and withdrawals. Instead, they will likely swap BTC for wBTC on L2s, previously bridged by professional operators and sold for a convenience fee⁶.

⁶This behavior is common on chains like Ethereum and Solana, with multi-billion annual rebalancing volumes.

8 Practicality, Limitations, and Extensions

We conclude by discussing key practical aspects of our design as well as limitations, design trade-offs, and extensions.

Funding challenge. Any user can challenge an operator’s Claim, forcing them to reveal the Assert transaction, which results in increased on-chain costs even for an honest operator. To prevent such griefing attacks, challengers must post collateral covering the fees of an Assert transaction.

While this collateral protects operators, it may be costly for a single challenger to provide. To address this, we allow the challenge collateral to be crowdfunded to amortize costs. The operator presigns the first input and output of Challenge using the SIGHASH flag [32] `SIGHASH_SINGLE|ANYONECANPAY`, enabling multiple challengers to contribute inputs until the required collateral is reached.

Operator liquidity. By design, when a user initiates a peg-out, one of the operators must front the equivalent BTC to the user immediately, before that operator can reclaim the locked funds from the Bitcoin side. The upside of this design is that users receive their BTC payouts without waiting for the dispute period to elapse – the withdrawal is effectively instant from the user’s perspective. The downside is that operators need sufficient BTC liquidity on hand to cover user withdrawals on the spot. This introduces a capital cost for operators, who must tie up funds to serve the bridge. To compensate, the protocol includes operator fees (and potentially additional incentives from the sidesystem, such as a share of L2 transaction fees) so that operating the bridge remains profitable despite the liquidity demands. This liquidity-provider model is already common in cross-chain bridges, and such incentives ensure that operators remain willing to serve exits. Future additions to Bitcoin’s consensus could allow for a design in which the identity of the spender of the PegOut transaction does not need to be determined at the time of setup, removing the need for operators altogether.

Large, non-standard transactions. Some transactions, notably Assert and Disprove, exceed Bitcoin’s 400 kB “standardness” relay policy. While this prevents them from propagating through the default peer-to-peer network, miners can still include them if received directly. Several mining pools already provide public submission channels for valid non-standard transactions,⁷ making inclusion feasible in practice. We discuss how we could reduce the transaction sizes in Section 8.

Fixed deposit amounts. Our current BITVM2-CORE instantiation only supports fixed deposit sizes, meaning users must hold the exact wrapped amount to peg out. In practice, this can be mitigated by deploying multiple bridge instances with different denominations (e.g., 0.5–100 BTC). We also expect peg-ins/outs to be handled by professional liquidity providers who aggregate and rebalance deposits on behalf of users.

⁷See, e.g., <https://slipstream.mara.com/>.

Covenant emulation via a signer committee. One of the limitations of BITVM2-CORE stems from the need to emulate covenants using a signer committee. Since we only use n -of- n multisignatures, i.e., without threshold signing, the signer set can be scaled far beyond the “vanilla” Bitcoin multisignature to 100+ signers using protocols like MuSig [25, 27, 28], at the cost of increased off-chain communication and coordination effort. While the implementation of this setup is beyond the scope of this paper, we note that such setups have been successfully executed at a much larger scale, for example, Ethereum’s KZG trusted setup ceremony⁸ which involved over 140,000 participants, executed via a simple website.

For each bridge peg-in instance, we assume that the signer committee behaves honestly and participates in the BITVM setup protocol. This leads to a fundamental design choice: one may either reuse the same signer committee across multiple bridge instances, requiring the signers to remain continuously available, or instantiate a fresh signer set per bridge instance, which only needs to be honest and active during its one-time setup.

Signature optimization. Replacing Lamport with Winternitz signatures [10] halves the signature size.

Tuning Assert and Disprove Sizes. The current design commits to the full SNARK execution trace in a single Assert transaction. In principle, the computation could be split across multiple Assert transactions that could be used in parallel, each committing to a smaller execution segment, thereby reducing the size of individual Assert and Disprove transactions. However, under covenant emulation, this approach incurs a fundamental trade-off: with n parallel Assert segments, the signer committee must presign $O(2^n)$ Payout transactions to ensure an honest operator can reclaim funds, regardless of which of the n Assert transactions a malicious challenger forces them to reveal.

Reducing cost of happy path. We can reduce the cost of the happy path, by moving the AssertScript from Claim to a transaction that sits between Claim and Assert, thus making Claim significantly smaller.

Atomic swaps. To address both fixed deposit sizes and the requirement to run additional software for initiating a BITVM2-CORE deposit, we expect that peg-ins and peg-outs in BITVM2-BRIDGE be handled primarily by professional users offering these operations as a service. In particular, bridge operators may naturally assume this role and act as market makers. Under this model, end users do not interact with peg-in and peg-out transactions directly. Instead, users enter and exit the sidesystem via cross-chain swaps [18]. A market maker (Alice) maintains balances in both BTC on Bitcoin and wBTC on the sidesystem, and offers users (e.g., Bob) the ability to swap between the two assets against a fee. The market maker periodically rebalances her inventories

by executing peg-in and peg-out operations on BITVM2-BRIDGE as needed. This design abstracts away denomination constraints and protocol complexity from end users, while concentrating bridge interactions on actors equipped to manage liquidity and operational overhead. Similar architectures have been deployed at scale on Ethereum and its layer-2 systems via so-called “liquidity bridges.”⁹

Rotating operators. In our design, we assume that each BITVM2-BRIDGE instance has a pre-defined operator set. While it is possible to add and remove operators during runtime, the implementation of a such mechanism depends on the Covenant model. In our case, where we emulate covenants via a signer committee, the m signers cannot add or remove operators later on, as this would contradict our protocol and model that assumes at least one honest signer that deletes their key. Even if such modifications were possible, for example, by employing key-evolving cryptography, our signer set is designed to be large for safety reasons. As a result, the coordination overhead may be significant and the failure rate (e.g. a signer being offline) high, possibly making ad-hoc operator rotation impractical for an existing bridge instance.

Instead, we can introduce a pre-defined, periodic rotation schedule embedded in the BITVM2-BRIDGE ruleset, i.e., the SNARK verifier. Under this model, as long as there exists one honest operator per BITVM2-BRIDGE instance, we could rotate operators for existing instances by executing a peg-out transaction that at the same time acts as a peg-in transaction for a new BITVM2-BRIDGE instance with a different operator set. A similar rotation design is implemented by tBTCv2¹⁰.

The design of mechanisms for selecting operators for a BITVM2-BRIDGE instance is orthogonal to our construction and outside the scope of this paper. Existing approaches include random sampling based on staked assets, as commonly employed in Proof-of-Stake systems (e.g., staking BTC [11, 24]), as well as sampling based on Proof-of-Work contributions over a fixed time window, for instance in sidesystems that are merge-mined with Bitcoin.

Alternative light client designs. Beyond the approach adopted in this work, one may consider alternative Bitcoin light client constructions based on explicit block commitments by operators.

One such design requires operators to periodically commit on-chain to the latest Bitcoin block they observe, using Lamport or Winternitz signatures that enable challenges against incorrect commitments. While this approach provides verifiability, it incurs substantial communication overhead and on-chain costs, making it impractical.

A refinement of this approach replaces periodic on-chain commitments with optimistic off-chain publication, for example via a bulletin board or a blockchain with inexpensive data

⁸<https://github.com/ethereum/kzg-ceremony>

⁹See, for example, <https://uniswap.org/whitepaper-uniswapx.pdf> and <https://docs.across.to/introduction/what-is-across>.

¹⁰<https://github.com/keep-network/tbtc-v2/tree/main/docs>

availability. This makes the optimistic case of full honesty cheaper. If an operator omits or publishes an incorrect commitment, a challenger can force the commitment on-chain, where it can be disputed using a protocol similar to Section 3.

However, in the worst case, a malicious challenger can repeatedly trigger on-chain commitments, effectively forcing all data on-chain. While one could require challengers to cover or share the resulting on-chain costs, the associated capital requirements undermine the practicality of this approach.

Consensus changes enhancing BITVM2. Various proposed consensus changes would enable more efficient bridge designs: (a) Native big-integer arithmetic [30] would substantially reduce the SNARK verifier’s script size from gigabytes to megabytes, potentially enabling full verification in a single transaction and significantly simplifying the design. (b) Native support for signature verification of arbitrary stack data (e.g., `OP_CHECKSIGFROMSTACK`) would reduce the cost of one-time signature schemes, improving the efficiency of Winternitz signatures by avoiding indirect message encoding. (c) Any covenant proposal that allows committing to transaction inputs would remove the need for a signer committee, guaranteeing unconditional deposits safety. Proposals include introspection opcodes (e.g., `OP_INSPECTINPUTOUTPUTPOINT`) or `TXHASH`, with introspection-based approaches being particularly efficient in terms of transaction fees.

Beyond rollups. In this work we focus on Bitcoin rollups, which naturally fit our design by using Bitcoin consensus through data commitments and verifiable state transitions. More generally, BITVM2-CORE can also support bridges to blockchains with independent consensus, by embedding their light client inside the SNARK verifier.

Acknowledgments

The work was partially supported by the European Research Council (ERC) under the European Union’s Horizon 2020 research (grant agreement 771527-BROWSEC), by the Austrian Science Fund (FWF) through the SFB SpyCode project F8510-N and F8512-N, by the Austrian Federal Ministry for Digital and Economic Affairs, the National Foundation for Research, Technology and Development and the Christian Doppler Research Association through the Christian Doppler Laboratory Blockchain Technologies for the Internet of Things (CDL-BOT), by the WWTF through the project 10.47379/ICT22045 and by Ministero dell’Università e della Ricerca (MUR), issue D.M. 352/2022 “Borse di Dottorato” - Dottorato di Ricerca di Interesse Nazionale in “Blockchain & Distributed Ledger Technology”, under the National Recovery and Resilience Plan (NRRP), by Input Output (<https://iohk.io>) through their funding of the Edinburgh Blockchain Technology Lab, and by the Stanford IOG Blockchain Research Hub. We also thank the members of the BitVM Alliance for implementing BitVM2 and Babylon for performing the mainnet

experiment (Section 7).

Ethical Considerations

We here discuss the ethical considerations of our work. Our contribution is twofold: BITVM2-CORE, a protocol for proving on Bitcoin the output of arbitrary functions, and BITVM2-BRIDGE, a protocol for transferring bitcoins to and from sidesystems. It is expected to more closely integrate Bitcoin with other blockchains and cryptocurrencies. As such, the relevant stakeholders are individuals and organizations that invest directly or indirectly in Bitcoin and other cryptocurrencies, entities that develop blockchain infrastructure and provide bridging services, and the public at large.

Many Bitcoin users choose to bridge their bitcoins to other systems for a variety of reasons. Our work improves the state of the art by building Bitcoin bridges with minimal trust to third parties. Such bridges lead to loss of funds under strictly fewer scenarios than alternatives, expanding the scope of beneficial outcomes of Bitcoin bridging. Reducing the risks of Bitcoin bridging can have second-order benefits, including the integration of Bitcoin, the most secure and established cryptocurrency, into complex smart contracts, an increase in Bitcoin price, and wider Bitcoin adoption, benefiting all stakeholders involved with Bitcoin.

Potential harm for bridge users may come in two forms: Firstly, if our trust assumptions fail, loss of funds is still possible. We consider this risk to be acceptable, as alternatives involve strictly higher risks of failure. Furthermore, different bridge users may choose to trust different sets of operators, meaning that a trust failure for one user does not necessarily lead to loss of funds for another user, i.e., trust failures are local. Secondly, the reduced trust requirements of our protocol may prompt currently passive Bitcoin holders that are unfamiliar with the risks of cryptocurrency investment vehicles of other blockchains to engage in risky investment strategies, potentially causing loss of savings. We judge that this portion of the population is sufficiently small, and that, in line with the Menlo report, sufficient harm reduction measures against reckless investment are in place, as well as that individuals have the right to take autonomous, informed decisions regarding their financial behavior, even if it leads to harm.

Our work involves no human subjects. Furthermore, we do not provide a complete end-user product. Still, we urge any entity that provides our protocol to end-users to respect the autonomy of persons, while providing sufficient protections against harmful use, e.g., reckless investment decisions.

Bitcoin does not offer financial privacy and our work does not change the status quo in this respect. Future end-users should be informed that third parties may potentially deduce the users’ financial history by observing Bitcoin and the relevant sidesystem.

In the interest of transparency, all outcomes of our work, including code implementations and experimental data, are

made publicly available. We refer the reader to our statement on Open Science for more details.

Open Science

Our full implementation of BitVM2 is available at <https://doi.org/10.5281/zenodo.17949747>. The transactions involved in the on-chain validation of the unhappy path, which was conducted on June 3, 2025, are the following: Claim [4], Challenge [3], Assert [2] and Disprove [5] — we provide their transaction IDs and links to them in a blockchain explorer webpage in the references.

References

- [1] BitVM reference implementation: Zenodo code repository. <https://doi.org/10.5281/zenodo.17949747>.
- [2] Bitcoin transaction 32129c9...08711f. <https://mempool.space/tx/32129c9ba93f5cf69fe135c3108f869b80b3370dbbcaf22ca33ed309cb08711f>, 2025. Accessed 5 Jun 2025.
- [3] Bitcoin transaction 49f3425...5d80581. <https://mempool.space/tx/49f3425b595304c4d79d0b890837c21c76c518df87ef403f99faeaa6a5d80581>, 2025. Accessed 5 Jun 2025.
- [4] Bitcoin transaction 6b096f3...d4d147a. <https://mempool.space/tx/6b096f35814cf26c3df031b22d91dfda56d25356f00c07d89cdebff71d4d147a>, 2025. Accessed 5 Jun 2025.
- [5] Bitcoin transaction 8ecfef...510249. <https://mempool.space/tx/8ecfef4d38a229c7cea10f6973f49d8bbe3a620fd557f7f2cb3658ac59510249>, 2025. Accessed 5 Jun 2025.
- [6] Lukas Aumayr, Zeta Avarikioti, Robin Linus, Matteo Maffei, Andrea Pelosi, Christos Stefo, and Alexei Zamyatin. BitVM: Quasi-turing complete computation on bitcoin. Cryptology ePrint Archive, Paper 2024/1995, 2024.
- [7] Lukas Aumayr, Zeta Avarikioti, Matteo Maffei, Giulia Scaffino, and Dionysis Zindros. Blink: An optimal proof of proof-of-work. In *Financial Cryptography*, 2025.
- [8] Adam Back, Matt Corallo, Luke Dashjr, Mark Friedenbach, Gregory Maxwell, Andrew Miller, Andrew Poelstra, Jorge Timón, and Pieter Wuille. Enabling blockchain innovations with pegged sidechains, 2014. <https://blockstream.com/sidechains.pdf>.
- [9] Federico Barbacovi and Enrique Larraia. Enforcing arbitrary constraints on bitcoin transactions. Cryptology ePrint Archive, Paper 2025/912, 2025.
- [10] Dan Boneh and Victor Shoup. A graduate course in applied cryptography. *Draft 0.6*, 2023.
- [11] Xinshu Dong, Orfeas Stefanos Thyfronitis Litos, Ertem Nusret Tas, David Tse, Robin Linus Woll, Lei Yang, and Mingchao Yu. Remote staking with economic safety. *arXiv preprint arXiv:2408.01896*, 2024.
- [12] Ariel Futoransky, Fadi Barbara, Ramses Fernandez, Gabriel Larotonda, and Sergio Demian Lerner. TOOP: A transfer of ownership protocol over bitcoin. Cryptology ePrint Archive, Paper 2025/964, 2025.
- [13] Juan Garay, Aggelos Kiayias, and Nikos Leonardos. The bitcoin backbone protocol: Analysis and applications. In *Advances in Cryptology-EUROCRYPT 2015*, pages 281–310. Springer, 2015.
- [14] Craig Gentry and Daniel Wichs. Separating succinct non-interactive arguments from all falsifiable assumptions. In *Proceedings of the Forty-Third Annual ACM Symposium on Theory of Computing*, STOC ’11, 2011.
- [15] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing*, 17(2):281–28, 04 1988. Copyright - Copyright] © 1988 Society for Industrial and Applied Mathematics; Last updated - 2023-12-04.
- [16] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology-EUROCRYPT*, pages 305–326. Springer, 2016.
- [17] Maurice Herlihy. Atomic cross-chain swaps. In *Proceedings of the 2018 ACM Symposium on Principles of Distributed Computing*, PODC ’18, page 245–254, New York, NY, USA, 2018. Association for Computing Machinery.
- [18] Maurice Herlihy. Atomic cross-chain swaps. arXiv:1801.09515, 2018. Accessed:2018-01-31.
- [19] Victor I. Kolobov, Avihu M. Levy, and Moni Naor. ColliderVM: Stateful computation on bitcoin without fraud proofs. Cryptology ePrint Archive, Paper 2025/591, 2025.
- [20] Misha Komarov. Bitcoin pipes: Covenants on bitcoin without soft fork. <https://delvingbitcoin.org/t/bitcoin-pipes-covenants-on-bitcoin-without-soft-fork/1195>, 2024.
- [21] Jae Kwon and Ethan Buchman. Cosmos whitepaper. *A Netw. Distrib. Ledgers*, 27:1–32, 2019.

- [22] Leslie Lamport. Constructing digital signatures from a one way function. Technical Report CSL-98, October 1979.
- [23] Sergio Demian Lerner, Ramon Amela, Shreemoy Mishra, Martin Jonas, and Javier Álvarez Cid-Fuentes. Bitvmx: A cpu for universal computation on bitcoin, 2024.
- [24] Robin Linus. Stakechain: A bitcoin-backed proof-of-stake. In *International Conference on Financial Cryptography and Data Security*, pages 3–14. Springer, 2022.
- [25] Gregory Maxwell, Andrew Poelstra, Yannick Seurin, and Pieter Wuille. Simple schnorr multi-signatures with applications to bitcoin. *Designs, Codes and Cryptography*, 87(9):2139–2164, 2019.
- [26] Threshold Network. tbtc: Threshold network documentation. <https://docs.threshold.network/>, 2020. Accessed: 2025-09-19.
- [27] Jonas Nick, Tim Ruffing, and Yannick Seurin. Musig2: Simple two-round schnorr multi-signatures. In *Annual International Cryptology Conference*, pages 189–221. Springer, 2021.
- [28] Jonas Nick, Tim Ruffing, Yannick Seurin, and Pieter Wuille. Musig-dn: Schnorr multi-signatures with verifiably deterministic nonces. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 1717–1731, 2020.
- [29] Joseph Poon and Thaddeus Dryja. The bitcoin lightning network. Whitepaper, 2015.
- [30] Rusty Russell. The great script restoration, jan 2024. <https://github.com/bitcoin/bips/blob/c2f268e83031b9b67e798c5c72a1171bfc463d1f/bip-unknown-var-budget-script.mediawiki>.
- [31] Sri AravindaKrishnan Thyagarajan, Giulio Malavolta, and Pedro Moreno-Sanchez. Universal atomic swaps: Secure exchange of coins across all blockchains. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 1299–1316, 2022.
- [32] Bitcoin Wiki. Contract: Sighash flags, sep 2023.
- [33] Pieter Wuille, Jonas Nick, and Anthony Towns. Bip 0341, taproot: Segwit version 1 spending rules, jan 2020.
- [34] Tiancheng Xie, Jiaheng Zhang, Zerui Cheng, Fan Zhang, Yupeng Zhang, Yongzheng Jia, Dan Boneh, and Dawn Song. zkbridge: Trustless cross-chain bridges made practical. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS '22*, 2022.

- [35] Alexei Zamyatin, Mustafa Al-Bassam, Dionysis Zindros, Eleftherios Kokoris-Kogias, Pedro Moreno-Sanchez, Aggelos Kiayias, and William J Knottenbelt. Sok: Communication across distributed ledgers, 2019.

- [36] ZeroSync. BitVM Github repository, dec 2023. <https://github.com/BitVM/BitVM>.

A Covenant Emulation

In this section, we formally define covenant emulation and prove that the method described at a high level in Section 3, “Signer Committee” Subsection (and its algorithm, described in Appendix E) implements this interface. We first need some preliminary definitions.

Definition A.1 (Spending Tree). Given a transaction skeleton $Tx := ([in_1, \dots, in_n], [out_1, \dots, out_m])$, where $out_i := (a_i \text{ BTC}, lockScript_i)$ for all i , we define the *spending tree* of Tx as the tree T of transaction skeletons, where:

- (i) Tx is the root of T ,
- (ii) each child node of Tx is a transaction that spends (one of) the output(s) locking script, for any possible a and $lockScript$,
- (iii) each child node is the root of a spending tree.

In a spending tree with root Tx , we call *spending path* between Tx and an intermediate node Tx' the nodes on the path from Tx to Tx' . By writing $Tx < Tx'$, we denote that there is a spending path between Tx and Tx' , and by $Tx \not< Tx'$ we denote that no such path exists.

Definition A.2 (Covenant Emulation System). Given a transaction skeleton Tx , a *covenant emulation system (CES)* for Tx is a protocol that, on input Tx and a security parameter λ , outputs a tree T' with root Tx , where T' that is a subtree of the spending tree T .

We refer to the tree output of a CES protocol as *covenant tree*. We denote the i -th leaf of the covenant tree T' as $T'.leaf(i)$. A CES is called *restrictive* if, given its output covenant tree, a polynomially bounded adversary cannot publish on-chain a transaction that is in the spending path of the root but not in the spending path of any leaf of the covenant tree, unless with negligible probability.

Definition A.3 (Restrictiveness). A CES is *restrictive* if, for all PPT $\mathcal{A}$, for any execution of the ledger protocol Π_L , it holds that for any round r after the CES protocol execution:

$$\Pr[Tx' \in {}^r\mathcal{L} \cap : Tx < Tx' \wedge \forall i, T'.leaf(i) \not< Tx' \mid T' \leftarrow CES(\lambda, Tx)] \leq \text{negl}(\lambda). \quad (1)$$

In the Setup procedure of the BITVM2-CORE protocol (cf. Algorithm E.1), we implement a Covenant Emulation

System. The BITVM2-CORE prover collaborates with a set of signers to create a covenant tree to condition the spending of the Claim transaction. Specifically, the prover creates the transaction skeletons that are the nodes of the covenant tree with root Claim, and sends them to the signers. They receive the transactions and sign some of them only if the covenant tree is constructed as expected. The resulting covenant tree is shown in Fig. 10.

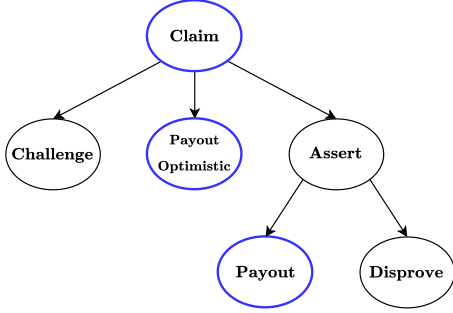


Figure 10: The covenant tree for the Claim transaction built by the Setup procedure of the BITVM2-CORE protocol. Transactions depicted with a blue border are signed by the signing committee during protocol execution.

Lemma A.1. *If the signature scheme Σ is EUF-CMA secure and there is an honest signer, then the procedure described in Algorithm E.1 is a restrictive Covenant Emulation System.*

We defer the proof to Appendix F.

B Background and Notation

B.1 Lamport digital signature scheme

Let $h : X \rightarrow Y$ be a one-way function, where $X := \{0, 1\}^*$ and $Y := \{0, 1\}^\lambda$, for a given security parameter λ . Let $m \in \{0, 1\}^\ell$ be a ℓ -bit message, with $\ell \in \mathbb{N}_{>0}$. A *Lamport digital signature scheme* [22] Lam consists of a triple of algorithms (KeyGen, Sig, Vrfy), where:

- $(pk_{\mathcal{M}}, sk_{\mathcal{M}}) \leftarrow \text{Lamp.KeyGen}(\ell)$ (Algorithm 4), is a PPT algorithm that takes as input a positive integer ℓ and returns a key pair, consisting of a secret key $sk_{\mathcal{M}}$ and a public key $pk_{\mathcal{M}}$ which can be used for one-time signing a ℓ -bit message. For readability, $\mathcal{M} = \{0, 1\}^\ell$ is an alias for the ℓ -bit message space.
- $c_m \leftarrow \text{Lamp.Sig}_{sk_{\mathcal{M}}}(m)$ (Algorithm 5), is a DPT algorithm parameterized by a secret key $sk_{\mathcal{M}}$, that takes as input a message $m \in \mathcal{M}$ and outputs the signature c_m , which we also call (Lamport) commitment.

- $\{\text{True}, \text{False}\} \leftarrow \text{Lamp.Vrfy}_{pk_{\mathcal{M}}}(m, c_m)$ (Algorithm 6), is a DPT algorithm parameterized by a public key $pk_{\mathcal{M}}$ that takes as input a message m , a signature c_m , and outputs True iff c_m is a valid signature for m generated by the secret key $sk_{\mathcal{M}}$, corresponding to $pk_{\mathcal{M}}$, i.e., $(pk_{\mathcal{M}}, sk_{\mathcal{M}})$ is a key pair generated by Lamp.KeyGen .

Algorithm 4 The key generation algorithm Lamp.KeyGen for a ℓ -bit messages space, which we shall call $\mathcal{M}$. Throughout these algorithms, we use matrix notation, i.e., $a[i, j]$ refers to the element at row i and column j of it.

```

1: function  $\text{Lamp.KeyGen}(\ell)$ 
2:   Let  $sk_{\mathcal{M}} \leftarrow \begin{pmatrix} x[0, 0], \dots, x[0, \ell - 1] \\ x[1, 0], \dots, x[1, \ell - 1] \end{pmatrix}$ , where every element  $x[i, j]$  is sampled uniformly from the set  $X$ ;
3:   for  $i = 0, 1$  and  $j = 0, \dots, \ell - 1$  do
4:      $y[i, j] \leftarrow h(x[i, j])$ ;
5:   end for
6:   Let  $pk_{\mathcal{M}} \leftarrow \begin{pmatrix} y[0, 0], \dots, y[0, \ell - 1] \\ y[1, 0], \dots, y[1, \ell - 1] \end{pmatrix}$ ;
7:   return  $(sk_{\mathcal{M}}, pk_{\mathcal{M}})$ .
8: end function
  
```

Algorithm 5 The Lamport signature algorithm Lamp.Sig , parameterized over a secret key $sk_{\mathcal{M}}$ for a ℓ -bit sized message space $\mathcal{M}$.

```

1: function  $\text{LampSig}_{sk_{\mathcal{M}}}(m)$ 
2:   for  $i = 0, \dots, \ell - 1$  do
3:     Let  $c_m[i] \leftarrow sk_{\mathcal{M}}[m[i], i]$ ;
4:   end for
5:   return  $c_m$ .
6: end function
  
```

Lamport signatures are secure one-time signatures. We write $sk_{\mathcal{M}}$ and $pk_{\mathcal{M}}$ to denote the secret key and the public key associated with the message space $\mathcal{M}$. This key pair can be used to sign any message in $\mathcal{M}$, but once signature c_m is created, the key pair is committed to one specific message $m \in \mathcal{M}$. In other words, as long as only one message $m \in \mathcal{M}$ is signed with a single key pair, no polynomially bounded adversary will be able to forge a signature over another message $m' \neq m$ for that key pair with non-negligible probability.

More concretely, when a party signs a message using a Lamport signature scheme, they reveal, for every bit $m[i]$ of the message, one of the two preimages $x[0, i]$ and $x[1, i]$, with $i = 0, \dots, \ell - 1$. This means that the signer is claiming that $m[i]$ is either 0 or 1. Notice that committing to an ℓ -bit message m is just the same as making ℓ commitments to 1-bit messages (one for each bit of m).

Algorithm 6 The Lamport verification algorithm Lamp.Vrfy , parameterized over a public key $pk_{\mathcal{M}}$ for a ℓ -bit message space $\mathcal{M}$.

```

1: function  $\text{Lamp.Vrfy}_{pk_{\mathcal{M}}}(m, c_m)$ 
2:   for  $i = 0, \dots, \ell - 1$  do
3:     if  $h(c_m[i]) \neq pk_{\mathcal{M}}[m[i], i]$  then
4:       return False;
5:     end if
6:   end for
7:   return True.
8: end function

```

For a formal definition of one-time security and the proof that Lamport signatures are one-time secure digital signature schemes (assuming the existence of one-way functions), refer to, e.g., [10]. Lamport signatures, and in particular Algorithm 6, are implementable in Bitcoin Script; a sample implementation is available at [36].

Since we leverage Lamport signatures to enable a party to commit to a bit (and thus to a message), we refer to the Algorithm 5 as LampComm instead of Lamp.Sig and to the Algorithm 6 as CheckLampComm instead of Lamp.Vrfy .

B.2 Transactions in the UTXO model

We identify a user U on a ledger L by the key pair (pk_U, sk_U) of a signature scheme Σ , used to prove ownership over coins. We let $\sigma_U(m)$ be the digital signature of U over a message $m \in \{0, 1\}^*$. If it is clear what message is signed, we sometimes use σ_U as shorthand.

In the *unspent transaction output* (UTXO) model, each transaction output is associated with a coin value (in BTC). An output is defined as an attribute tuple $\text{out} := (a \text{ BTC}, \text{lockScript})$, i.e., it consists of an amount $\text{out}.a \in \mathbb{R}_{\geq 0}$ of coins BTC and the condition(s) out.lockScript under which it can be spent. A transaction Tx maps a non-empty list of existing, unspent outputs, to a non-empty list of newly created Tx.outputs . To distinguish them, we refer to the former as Tx.inputs of the transaction. An input, $\text{in} := (\text{PrevTx}, \text{outIndex}, \text{lockScript})$, uniquely identifies one existing output by referencing a transaction PrevTx and an output index outIndex , and is repeating the output's spending condition lockScript for convenience. We will refer to the lists of inputs and outputs of a transaction as *transaction skeleton*.

Formally, we define a transaction as $\text{Tx} := (\text{inputs}, \text{witnesses}, \text{outputs})$, which besides the aforementioned inputs $\text{Tx.inputs} := [in_1, \dots, in_n]$ and outputs $\text{Tx.outputs} := [out_1, \dots, out_m]$ contains the witness data, $\text{Tx.witnesses} := [w_1, \dots, w_n]$, which is the list of the tuples that fulfill the spending conditions of the inputs of the transaction, one witness for each input. The locking script,

expressed in the ledger's scripting language, is executed with the corresponding witness as script input. If this execution returns False, the transaction is invalid. If it returns True, the spending condition is fulfilled.

A transaction is valid when all witnesses fulfill the locking condition of their corresponding input; all of the transaction's inputs are unspent; the sum of the value of the outputs is smaller or equal to the sum of the value of the inputs (the difference is given to the miners).

Transaction spending conditions. We are particularly interested in Bitcoin, which has a stack-based scripting language. We now describe a subset of spending conditions supported on Bitcoin that are used in this paper. Each of the following can be combined using logical operators $\wedge$ (and), $\vee$ (or) to create more complex spending conditions.

- **Signature locks.** An output locked with CheckSig_{pk_U} can only be spent if the spending transaction is signed with the secret key of the key pair (sk_U, pk_U) .
- **Multisignature locks.** To fulfill a multisignature spending condition, a certain number k out of n signatures are required. For example, for users A and B , a 2-of-2 multisignature spending condition is denoted as $\text{CheckMultiSig}_{pk_{A,B}}$ and the respective signature as $\sigma_{A,B}$.
- **Timelocks** lock a transaction output until a specified time in the future (absolute timelock) or until a specific time after the transaction is included on-chain (relative timelock). We denote the former as $\text{AbsTimelock}(\Delta)$, and the latter as $\text{RelTimelock}(\Delta)$. In the following, we use timelocks in conjunction with other spending conditions. For instance, if the UTXO Tx.out_1 has locking script $\text{lockScript} := \text{RelTimelock}(\Delta) \wedge \text{CheckSig}_{pk_U}$, the user U can spend the UTXO Tx.out_1 after that a certain amount of time T has passed from the moment that Tx.out_1 has been published on-chain.
- **Taproot trees** [33], or Taptrees, make a UTXO spendable by satisfying one among multiple spending conditions. The spending conditions are (Tap)leaves of a Merkle tree. To spend a UTXO that has a Taptree as a locking script, a user needs to provide a witness for one of the leaves along with a Merkle inclusion proof of such leaf into the Taptree. We denote the Tapleaves of a Taptree locking script as $\langle \text{leaf}_1, \dots, \text{leaf}_r \rangle$; when a user fulfills script leaf_i to unlock the j -th UTXO of the transaction Tx , we write the corresponding input as $(\text{Tx}, j, \langle \text{leaf}_i \rangle)$. Every time that a user spends a UTXO via a Tapleaf of a Taptree, we assume that the user has provided a valid Merkle inclusion proof for the Tapleaf.
- **Other conditions.** We denote with True a condition that is always fulfilled and with False a condition that can never be fulfilled. In the latter case, the coins can not be redeemed, and they are *burnt* instead.

Additionally, we use $*$ to denote a transaction input, witness, or output that can be anything (valid according to Bitcoin consensus rules), but is irrelevant to our protocol.

B.2.1 SIGHASH flags

SIGHASH flags specify which part of the transaction data is included in the hash that is signed as part of a signature lock. These flags are primarily used to help coordinate multiple users in creating and signing a transaction.¹¹

- **ALL:** All inputs and outputs are signed. The transaction is only valid as is.
- **NONE:** All inputs are signed but no outputs. No constraints are imposed on where the funds go.
- **SINGLE:** All inputs but only one output are signed, i.e., other outputs can be added arbitrarily.

In addition, to create more advanced constructions, the ANYONECANPAY flag can be combined with any of the other flags to sign only one input instead of all inputs.

B.2.2 Connector outputs

*Connector outputs*¹² is a technique to ensure that only one of a given set of Bitcoin transactions is valid and can be included in the Bitcoin blockchain. Specifically, we create multiple transactions that require as input the same connector output. We achieve this by introducing an additional input (that references the connector output) to each transaction and set the SIGHASH_ALL flag, requiring all inputs to be present for the transaction to be valid. As soon as one of these transactions is included on-chain, spending the connector output, the other transactions become invalid. The connector output can thereby specify a variety of custom spending conditions.

C Cryptographic primitives

C.1 Digital signatures

A digital signature scheme Σ is a tuple of three algorithms: KeyGen, Sign, and Vrfy.

- $(pk, sk) \leftarrow \Sigma.\text{KeyGen}(1^\lambda)$ is a probabilistic, polynomial time (PPT) algorithm that takes a security parameter λ as input and returns a key pair, consisting of a secret (or private) sk and a public key pk .
- $\sigma \leftarrow \Sigma.\text{Sign}(sk, m)$ is a PPT algorithm that takes as input a secret key sk and a message $m \in \{0, 1\}^*$, and outputs an authentication tag, or *signature*, σ .

¹¹See https://en.bitcoin.it/wiki/Contract#SIGHASH_flags. A helpful visualization can be found here: <https://tinyurl.com/mr2kshzd>

¹²For example, described in <https://tinyurl.com/2p566ynp>.

- $\{\text{True}, \text{False}\} \leftarrow \Sigma.\text{Vrfy}(pk, \sigma, m)$ is a deterministic, polynomial time (DPT) algorithm that takes as input a public key pk , a signature σ and a message $m \in \{0, 1\}^*$, outputs True iff σ is a valid signature for m generated by the secret key sk , corresponding to pk , i.e., (pk, sk) is a key pair generated by $\Sigma.\text{KeyGen}$. For consistency with Bitcoin Script, we will refer to this as $\text{CheckSig}_{pk}(\sigma)$, the message being the transaction.

In this work, we make use of signature schemes that are EUF-CMA secure [15].

C.2 Succinct non-interactive arguments of knowledge (SNARKs)

For this definition, we closely follow, e.g., [14, 16]. Let $\mathcal{R} \leftarrow \hat{\mathcal{R}}(\lambda)$ be a relation generator that takes as input a security parameter λ , and returns a polynomial time decidable binary relation $\mathcal{R}$. We denote ϕ as the statement and w as the witness for the pairs $(\phi, w) \in \mathcal{R}$. We define an efficient publicly verifiable non-interactive argument of knowledge for $\mathcal{R}$ as a tuple of three PPT algorithms: Setup, Prove, and Vrfy.

- $crs \leftarrow \text{SNARK.Setup}(\mathcal{R})$ takes as input a relation $\mathcal{R}$, and outputs a common reference string crs .
- $\pi \leftarrow \text{SNARK.Prove}(\mathcal{R}, crs, \phi, w)$ takes as input a common reference string crs along with $(\phi, w) \in \mathcal{R}$, and returns an argument π .
- $\{\text{True}, \text{False}\} \leftarrow \text{SNARK.Vrfy}(\mathcal{R}, crs, \phi, \pi)$ takes as input a common reference string crs , a statement ϕ , and an argument π and returns True or False, (informally) depending on whether or not π is a valid argument.

$(\text{Setup}, \text{Prove}, \text{Vrfy})$ is a non-interactive argument of knowledge for $\mathcal{R}$ if it has *perfect completeness* and *knowledge soundness*, as defined in [14, 16]. On a high level, the former means that given any true statement ϕ , an honest prover can convince an honest verifier with overwhelming probability; the latter means that if a prover can convince an honest verifier, the prover must know a valid witness w , also with overwhelming probability. Finally, a non-interactive argument, where the verifier runs in polynomial time in $\lambda + |\phi|$ and the proof size is polynomial in λ , is denoted as (preprocessing) succinct non-interactive argument of knowledge (SNARK). The implementation of our scheme makes use of [16]; we are not interested in nor do we utilize the zero-knowledge property.

D Formal Model and Definitions

In this section, we introduce the notation, definitions, and security properties required to establish the security of our three protocols: BITVM2-CORE, the light client, and the BITVM2-BRIDGE.

Notation. We use ε and $[]$ to mean the empty string and empty sequence. By $x \parallel y$, we mean the string concatenation of x and y encoded in a way that x and y can be unambiguously retrieved. We denote by $|C|$ the length of the sequence C ; by $C[i]$ the i^{th} (zero-based) element of the sequence, and by $C[-i]$ the i^{th} element from the end. We use $C[i:j]$ to mean the subarray of C from the i^{th} element (inclusive) to the j^{th} element (exclusive). Omitting i takes the sequence to the beginning, and omitting j takes the sequence to the end. Given two sequences C_1, C_2 , we use $C_1 \leq C_2$ to mean that C_1 is a prefix of C_2 . Given a set A , we denote its power set by $\wp(A)$.

D.1 Protocol execution model

Let Π be a protocol that runs between n parties in the set $\mathcal{P} := \{P_1, \dots, P_n\}$ ¹³, in the presence of an adversary $\mathcal{A}$, a special party that can corrupt up to t other parties. Corrupting a party P_i means learning their internal state and taking full control over them from the moment of corruption until the end of protocol execution. The execution of protocol Π is driven by a special program $\mathcal{Z}$ called *environment* that spawns parties $P_1, \dots, P_n$ and the adversary $\mathcal{A}$. All of the aforementioned programs can be thought of as interactive Turing machines (ITM). We use the notation $\{VIEW_{\Pi, \mathcal{A}, \mathcal{Z}}^{P_i, r, t, n}(z)\}_{z \in \{0,1\}^*}$ to denote the transcript of party P_i at round r of the execution of protocol Π run by environment $\mathcal{Z}$ w.r.t. an adversary $\mathcal{A}$.

D.2 Ledger and network model

We consider two ledgers, denoted $\mathcal{L}_A$ and $\mathcal{L}_B$, operated by respective ledger protocols $\Pi_{\mathcal{L}_A}$ and $\Pi_{\mathcal{L}_B}$. In our main application, $\mathcal{L}_A$ corresponds to Bitcoin, and $\mathcal{L}_B$ to a second-layer or alternative chain. Each ledger protocol maintains a sequence of confirmed transactions visible to participants known as full nodes. We model the ledger as a function ${}^r\mathcal{L}_X^P$, denoting the view of party P of ledger $\mathcal{L}_X$ at round r .

Definition D.1 (Safety). A distributed ledger protocol is *safe* if: (i) For any honest party P and any rounds $r_1 \leq r_2$, it holds that ${}^{r_1}\mathcal{L}^P \leq {}^{r_2}\mathcal{L}^P$ (*Self-Consistency*); and (ii) For any honest parties P_1, P_2 and any round r , it holds that either ${}^r\mathcal{L}^{P_1} \leq {}^r\mathcal{L}^{P_2}$ or ${}^r\mathcal{L}^{P_2} \leq {}^r\mathcal{L}^{P_1}$ (*View-Consistency*).

Definition D.2 (Liveness). A distributed ledger protocol execution, in a synchronous network, is *live*(u) if any transaction written to any honest party ledger at round r , appears in the ledger of all honest parties by round $r + u$, denoted as ${}^{r+u}\mathcal{L}^\cap$.

We assume a global round-based clock, shared across all parties and ledgers. Rounds abstract time into discrete steps. We assume a synchronous network, i.e., all messages broadcast to the network will be delivered to all parties within a known time bound Δ . Ledgers and blockchains are defined

as in the Bitcoin backbone model [13], which in particular operates in the q -bounded setting, used in the formal light client analysis (cf. Appendix G).

D.3 Formal definitions and properties

In the following, we expose the formal properties for the BITVM2-CORE, the on-chain light client for Bitcoin, and the BITVM2-BRIDGE.

D.3.1 BITVM2-CORE security properties

Definition D.3 (On-chain State Verification Protocol). A BITVM2-CORE proof system is a protocol between a prover P and a set of verifiers V , where $P, V \subseteq \mathcal{P}_f$ are full nodes of a ledger protocol $\Pi_{\mathcal{L}}(\cdot)$. BITVM2-CORE consists of a tuple of procedures (Setup, Prove, Verify) defined as follows:

- **Setup**($1^\lambda, \hat{\mathcal{R}}_f$) $\rightarrow (\mathcal{T}, \mathcal{S})$: Given a security parameter 1^λ and an NP-relation $\hat{\mathcal{R}}_f$ defined over f ¹⁴, output two sets $\mathcal{T}$ and $\mathcal{S} \subseteq \mathcal{T}$. Informally, $\mathcal{T}$ encodes the possible execution paths restricted at setup, while $\mathcal{S}$ encodes the payout transactions, in which the prover claims the deposit. For an honest prover, at least one transaction of the set $\mathcal{S}$ will appear.
- **Prove**($\hat{\mathcal{R}}_f, Y, w$) $\rightarrow \pi$: Given a statement Y and a witness w , output a proof π that $(Y, w) \in \hat{\mathcal{R}}_f$.¹⁵
- **Verify**($\hat{\mathcal{R}}_f, Y, \pi$) $\rightarrow \{\text{True}, \text{False}\}$: Given the proof π for the statement Y , output True if π is valid, otherwise output False.

An on-chain state verification protocol is called *robust* if it satisfies the properties of completeness and soundness stated below. Completeness expresses that an honest operator will be able to reclaim its deposit. In particular, assume that the setup is done correctly and an honest prover P knows a witness w s.t. $(Y, w) \in \hat{\mathcal{R}}_f$ at a round r . Then, with overwhelming probability, at least one transaction where the prover reclaims the deposit (included in $\mathcal{S}$) will appear on the ledger ${}^{r'}\mathcal{L}^\cap$ where $r' \geq r + u_c$. On the other hand, soundness guarantees that a malicious prover P that does not know a valid witness w s.t. $(Y, w) \in \hat{\mathcal{R}}_f$, cannot reclaim the deposit. To this end, none of the transactions in $\mathcal{S}$ will appear on the ledger ${}^r\mathcal{L}^\cap$ for any round r , except with a negligible probability.

Definition D.4 (Completeness). Parameterized by u_c , a BITVM2-CORE proof system is *complete* if, for all f and $\hat{\mathcal{R}}_f$, for all $Y \in \mathcal{L}_{\hat{\mathcal{R}}_f}$, for an honest prover P , and any execution

¹⁴The relation induced by the function f will be defined when instantiating a protocol that implements the BITVM2-CORE interface.

¹⁵The proof in our system is essentially the witness itself, but for capturing more generic cases where, for example, the proof hides the witness, we keep the definition more general.

¹³The number n of parties that participate in the execution of protocol Π is fixed, but possibly unknown.

of the ledger protocol $\Pi_{\mathcal{L}}$ the following holds. For any round r after the Setup, and every round $r' \geq r + u_c$:

$$\Pr[\exists tx \in \mathcal{S} : tx \in {}^{r'}\mathcal{L} \cap \text{Cond1}] \geq 1 - \text{negl}(\lambda) \quad (2)$$

where Cond1 denotes:

$$\begin{aligned} (\mathcal{T}, \mathcal{S}) &\leftarrow \text{Setup}(1^\lambda, \hat{\mathcal{R}}_f); \\ w &\in \{\text{VIEW}_{\Pi_{\mathcal{L}}, \mathcal{A}, \mathcal{Z}}^{P, r, t, n}(z)\}_{z \in \{0,1\}^*} \wedge \\ \text{Verify}(\hat{\mathcal{R}}_f, Y, \text{Prove}(\hat{\mathcal{R}}_f, Y, w)) &= \text{True} \end{aligned}$$

Definition D.5 (Soundness). A BITVM2-CORE proof system is *sound* if, for all PPT $\mathcal{A}$ in the q -bounded setting, for any execution of the ledger protocol $\Pi_{\mathcal{L}}$, for all f and $\hat{\mathcal{R}}_f$, and for all $Y \in \mathcal{L}_{\hat{\mathcal{R}}_f}$, it holds that for any round r after the Setup:

$$\Pr[\exists tx \in \mathcal{S} : tx \in {}^r\mathcal{L} \cap \text{Cond2}] \leq \text{negl}(\lambda) \quad (3)$$

where Cond2 denotes:

$$\begin{aligned} (\mathcal{T}, \mathcal{S}) &\leftarrow \text{Setup}(1^\lambda, \hat{\mathcal{R}}_f); \\ w &\notin \{\text{VIEW}_{\Pi_{\mathcal{L}}, \mathcal{A}, \mathcal{Z}}^{P, r, t, n}(z)\}_{z \in \{0,1\}^*} \wedge \\ \text{Verify}(\hat{\mathcal{R}}_f, Y, \text{Prove}(\hat{\mathcal{R}}_f, Y, w)) &= \text{True} \end{aligned}$$

D.3.2 Light client security properties

We now present the security definition of the light client protocol based on BITVM2-CORE.

Definition D.6 ((u, k) -Admissible Block at r). Consider $u, k \in \mathbb{N}$ (liveness and safety parameter, respectively, of a ledger protocol). A block B that, at round r , fulfils the following properties is an *admissible block at r* :

Safety : $B \in \mathcal{C}_{r+u}^\cup[: -k]$

Liveness : $B \notin \mathcal{C}_r^\cap[: -k]$

Let $(\text{Com}, \text{Open})$ be a computationally binding (but not necessarily hiding) commitment scheme.

Definition D.7 ((u, k) -Admissible Commitment at r). A commitment $c \leftarrow \text{Com}(B)$ is (u, k) -admissible at r , if B is a (u, k) -admissible block at r :

Once it becomes true, a *monotonic ledger predicate* stays true even as the ledger grows:

Definition D.8 (Monotonic Ledger Predicate). We say that a predicate on a ledger is *monotonic* if $[p(\mathcal{L}_1) = 1 \wedge p(\mathcal{L}_2) = 0] \Rightarrow [\mathcal{L}_1 \text{ is not a prefix of } \mathcal{L}_2]$.

Definition D.9 (Predicate Light Client). An interactive Prover-Verifier protocol $\Pi(\mathcal{P}, V(p))$ is a *light client* for a monotonic ledger predicate p with safety parameter v if, when the protocol terminates at r , V outputs $p(\mathcal{L})$ for a ledger $\mathcal{L}$ such that:

- *Liveness*: ${}^r\mathcal{L} \cap$ is a prefix of $\mathcal{L}$,
- *Safety*: $\forall r' \geq r + v, \mathcal{L}$ is a prefix of ${}^{r'}\mathcal{L} \cup$.

Definition D.10 (Chain Client Security). An interactive Prover-Verifier protocol $\Pi(\mathcal{P}, V)$ is *secure* if, when the protocol terminates at r^* , V outputs a block B that is *admissible* at $r \leq r^*$.

D.3.3 BITVM2-BRIDGE security properties

We formalize a bridge protocol that operates atop two ledger protocols, $\Pi_{\mathcal{L}_A}$ and $\Pi_{\mathcal{L}_B}$. We denote an execution instance of this protocol as I . A bridge conditions the execution of one or more transactions on $\mathcal{L}_B$ upon the inclusion of one or more transactions on $\mathcal{L}_A$, which we refer to as *cross-chain request*. We assume a global clock governs both $\Pi_{\mathcal{L}_A}$ and $\Pi_{\mathcal{L}_B}$ ¹⁶.

To consistently track assets across chains, we associate each peg-in event with a unique identifier id . This identifier links all transactions, across both ledgers, that originate from the same peg-in, allowing the bridge to reason about asset continuity and cross-chain state. Intuitively, id serves as a reference to the coins created on $\mathcal{L}_B$ as a result of locking assets on $\mathcal{L}_A$. We now formalize the bridge using id to identify related transactions across ledgers, and a relation $R(\text{id})$ to specify valid cross-chain dependencies. Recall that $\wp(S)$ denotes the power set of set S .

Definition D.11 (Unidirectional Bridge). Let $X \in \{A, B\}$ and $V_{\mathcal{L}_X, \text{id}}$ be the set of syntactically valid transactions w.r.t. assets with identifier id in $\Pi_{\mathcal{L}_X}$. Parameterized by a binary relation $R(\text{id}) \subseteq \wp(V_{\mathcal{L}_A, \text{id}}) \times \wp(V_{\mathcal{L}_B, \text{id}})$, a *bridge* $\Lambda_{R(\text{id})}(\Pi_{\mathcal{L}_A}, \Pi_{\mathcal{L}_B})$ is a protocol that relays requests (e.g., coin transfer) issued on a source ledger $\mathcal{L}_A$ to a destination ledger $\mathcal{L}_B$.

A bridge protocol is *robust* if it satisfies both *bridge liveness* and *bridge safety* in every execution I . Bridge liveness ensures that if a transaction from a designated set is included on the source ledger, a corresponding transaction will eventually appear on the destination ledger – guaranteeing that honest parties can successfully bridge their funds. Bridge safety requires that transactions on the destination ledger appear only if a corresponding transaction exists on the source, possibly with delay, preventing coins from being minted without a valid deposit and preserving the total supply.

Definition D.12 (Bridge Liveness). A bridge protocol $\Lambda_{R(\text{id})}(\Pi_{\mathcal{L}_A}, \Pi_{\mathcal{L}_B})$ is *live* with *liveness* $u_\ell \in \mathbb{N}$ during I if, for all honest parties P_1 of $\mathcal{L}_A$ and P_2 of $\mathcal{L}_B$ and for all rounds $r_1, r_2 \in I$ with $r_2 \geq r_1 + u_\ell$ we have that, for all $\mathcal{A}, \mathcal{B}$ such that $(\mathcal{A}, \mathcal{B}) \in R(\text{id})$, if $\text{tx} \in \mathcal{A}$ is in the ledger ${}^{r_1}\mathcal{L}_A^{P_1}$, then there is $\text{tx}' \in \mathcal{B}$ such that tx' is in the ledger ${}^{r_2}\mathcal{L}_B^{P_2}$ with overwhelming probability.

¹⁶Although block production rates may differ, we assume full nodes in both protocols can compare transaction inclusion times against the global clock.

Definition D.13 (Bridge Safety). A bridge protocol $\Lambda_{R(\text{id})}(\Pi_{\mathcal{L}_A}, \Pi_{\mathcal{L}_B})$ is *safe* with *safety* $u_s \in \mathbb{N}$ during I if, for all honest parties P_1 of $\mathcal{L}_A$ and P_2 of $\mathcal{L}_B$ and for all rounds $r_1, r_2 \in I$ with $r_2 \leq r_1 + u_s$ we have that, for all $\mathcal{A}, \mathcal{B}$ such that $(\mathcal{A}, \mathcal{B}) \in R(\text{id})$, if $\text{tx}' \in \mathcal{B}$ is in the ledger ${}^{r_2}\mathcal{L}_B^{P_2}$, then there is $\text{tx} \in \mathcal{A}$ such that tx is in the ledger ${}^{r_1}\mathcal{L}_A^{P_1}$ with overwhelming probability.

We call bridge $\Lambda(u_s, u_l)$ —**robust** during I , with $u_s, u_l \in \mathbb{N}$, if it is safe with safety u_s during I and live with liveness u_l during I . We say that a bridge Λ is (u_s, u_l) —robust if it is (u_s, u_l) —robust for every execution I with overwhelming probability.

We now formalize the notion of a wrapped asset bridge, which supports bidirectional asset transfers between two ledgers.

Definition D.14 (Wrapped Asset Bridge). A *wrapped asset bridge* between ledgers $\mathcal{L}_A$ and $\mathcal{L}_B$ is defined by two unilateral bridge instances, $\Lambda_{R_1}^{\text{PegIn}}(\Pi_{\mathcal{L}_A}, \Pi_{\mathcal{L}_B})$ and $\Lambda_{R_2}^{\text{PegOut}}(\Pi_{\mathcal{L}_B}, \Pi_{\mathcal{L}_A})$, both associated with a common asset identifier id . Each instance is parameterized by a relation, $R_1(\text{id}) \subseteq \wp(V_{\mathcal{L}_A}) \times \wp(V_{\mathcal{L}_B})$, $R_2(\text{id}) \subseteq \wp(V_{\mathcal{L}_A}) \times \wp(V_{\mathcal{L}_B})$, where $V_{\mathcal{L}_A}$ and $V_{\mathcal{L}_B}$ denote the sets of valid ledger states or transactions.

The relation $R_1(\text{id})$ captures *peg-in* behavior, where locking assets on $\mathcal{L}_A$ implies minting on $\mathcal{L}_B$. Similarly, $R_2(\text{id})$ captures *peg-out* behavior, where burning assets on $\mathcal{L}_B$ implies unlocking on $\mathcal{L}_A$.

We further define an additional desired property for our bridge, namely *Bridge Operator Safety* (Definition D.15), to ensure that operators will not lose money by correctly operating the bridge protocol (even against Byzantine adversaries). This property guarantees that whenever an honest operator fronts money to a client, they can later reclaim it.

To this end, we additionally use the notation (tx, p) to indicate that the transaction tx is related to operator p . Specifically, it is paid by p if $\text{tx} \in \text{PegOut}$, and it pays p if $\text{tx} \in \{\text{Payout}, \text{PayoutOptimistic}\}$.

Definition D.15 (Bridge Operator Safety). A bridge protocol $\Lambda_{R_1}^{\text{PegIn}}, \Lambda_{R_2}^{\text{PegOut}}$ is said to be *operator safe* with *recovery parameter* $u_\Lambda \in \mathbb{N}$ during the execution I if, for any honest operator p , the following holds:

If there is a transaction $\text{tx} \in \text{PegOut}(\text{id})$ such that (tx, p) is included in ${}^{r_1}\mathcal{L}_A^{P_1}$ for all honest parties P_1 at some round r_1 , then there must exist a transaction $\text{tx}' \in \{\text{Payout}, \text{PayoutOptimistic}\}$ such that for every round $r_2 \geq r_1 + u_\Lambda$, $(\text{tx}', p) \in {}^{r_2}\mathcal{L}_A^{P_1}$ for all honest P_1 .

E The BITVM2-CORE Protocol

Here we present the BITVM2-CORE protocol that implements the interface defined in Definition D.3. In addition to

the prover P and the set of verifiers $\{V_i\}_{i \leq n}$, a set of signers $SIG := \{\text{Signer}_i\}_{i \leq m}$ participate in the protocol. The prover, the verifiers, and the signers are full nodes of the ledger protocol $\Pi_{\mathcal{L}}(\cdot)$ (and, as such, have access to the methods READ and WRITE as defined in Appendix F).

The protocol internally uses a EUF-CMA secure signature scheme $\Sigma := (\text{KeyGen}, \text{Sign}, \text{Vrfy})$ as the ledger signature scheme. The protocol is parameterized over two positive integers k, ℓ . Participants are given the common reference string crs , which is the result of $\text{SNARK.Setup}(\mathcal{R})$ procedure. Moreover, participants in the protocol have access to the following subroutines:

- $\text{Chunk}(\Pi, k) \rightarrow (f_1, \dots, f_k)$: breaks a Bitcoin script program Π of length l in k chunks, each of length l/k ¹⁷.
- $\text{CreateTx}(\text{TxName}, [pk_1, \dots, pk_\ell])$: builds the transaction skeleton of TxName as specified by the corresponding equation described in Section 3. Additionally, it takes the optional argument(s) $pk_1, \dots, pk_\ell$ and hard-codes them in TxName 's locking script.
- $\text{Verify}(\mathcal{S}, \mathcal{T}) \rightarrow b$: returns True if the transactions in the sets $\mathcal{S}$ and $\mathcal{T}$ are constructed as expected, i.e., as specified by the corresponding equations in Section 3. Returns False otherwise.

In the protocol, we use the notation $m \hookrightarrow P$ to mean that the active party is sending the message m to party P , and $m \leftarrow P$ to mean that the party P is receiving message m . Moreover, the notation $\text{tx} \xrightarrow{\tau} \mathcal{L}$ means that the active party calls $\text{WRITE}(\cdot)$ method of the ledger protocol with input tx at round τ . The notation $\text{tx} \xleftarrow{\tau} \mathcal{L}$, indicates the event of a successful $\text{READ}(\text{tx})$ operation for the transaction tx at round τ . For brevity, when a party attempts to write a transaction on the ledger, we explicitly mention only the portion of the transaction witness that is relevant for the protocol logic. For a complete description of every transaction witness, refer to Section 3.

E.1 $\text{Setup}(1^\lambda, \mathcal{R}) \rightarrow (\mathcal{T}, \mathcal{S})$:

Signer_i

1. $(pk_{\text{sig}_i}, sk_{\text{sig}_i}) \leftarrow \Sigma.\text{KeyGen}(1^\lambda)$.
2. $pk_{\text{sig}_i} \hookrightarrow \text{Prover}$.

Upon $(\mathcal{S}, \mathcal{T}) \leftarrow \text{Prover}$:

1. If $\text{Verify}(\mathcal{S}, \mathcal{T})$ is True, run $\sigma_1 \leftarrow \Sigma.\text{Sign}(sk_{\text{sig}_i}, \text{Payout})$, $\sigma_2 \leftarrow \Sigma.\text{Sign}(sk_{\text{sig}_i}, \text{PayoutOptimistic})$, $\sigma_3 \leftarrow \Sigma.\text{Sign}(sk_{\text{sig}_i}, \text{Claim})$.
2. $(\sigma_1, \sigma_2, \sigma_3) \hookrightarrow \text{Prover}$.

¹⁷if $k \nmid l$, the last chunk will be shorter.

3. Delete secret key sk_{sig_i} .

Prover :

1. $(f_1, \dots, f_k) \leftarrow \text{Chunk}(\text{SNARK.Vrfy}, k)$.
2. For each $i = 0, \dots, k$, run $(sk_{z_i}, pk_{z_i}) \leftarrow \text{Lamp.KeyGen}(\ell)$.

Wait until $pk_{sig_i} \leftarrow \text{Signer}_i$ for all $\text{Signer}_i \in \text{SIG}$.

1. Run $\text{CreateTx}(\text{Payout}, pk_{sig_1}, \dots, pk_{sig_m})$,
 $\text{CreateTx}(\text{PayoutOptimistic}, pk_{sig_1}, \dots, pk_{sig_m})$,
 $\text{CreateTx}(\text{Claim}, pk_{sig_1}, \dots, pk_{sig_m}, pk_{z_0}, \dots, pk_{z_k})$,
 $\text{CreateTx}(\text{Assert}, pk_{z_0}, \dots, pk_{z_k})$, $\text{CreateTx}(\text{Challenge})$,
 $\text{CreateTx}(\text{Disprove})$.
2. Let $\mathcal{S} := \{\text{Payout}, \text{PayoutOptimistic}\}$ and $\mathcal{T} := \mathcal{S} \cup \{\text{Claim}, \text{Assert}, \text{Challenge}, \text{Disprove}\}$.
3. For each $i = 1, \dots, m$, $(\mathcal{S}, \mathcal{T}) \hookrightarrow \text{Signer}_i$.

Wait until $(\sigma_1^i, \sigma_2^i, \sigma_3^i) \leftarrow \text{Signer}_i$ for all $\text{Signer}_i \in \text{SIG}$.

1. Let Payout' be the Payout transaction with the signatures $\sigma_1^1, \dots, \sigma_1^m$ added as part of the transaction witness. Define $\text{PayoutOptimistic}'$ and Claim' likewise. Let $\mathcal{S}' := \{\text{Payout}', \text{PayoutOptimistic}'\}$ and $\mathcal{T}' := \mathcal{S}' \cup \{\text{Claim}', \text{Assert}, \text{Challenge}, \text{Disprove}\}$.
2. Return $(\mathcal{S}', \mathcal{T}')$.

E.2 Prove($\mathcal{R}, Y, w$) $\rightarrow \pi$:

Prover :

1. Run $\text{SNARK.Prove}(\mathcal{R}, \text{crs}, Y, w) \rightarrow \pi$.
2. Let $z_0 := (\mathcal{R}, \text{crs}, Y, \pi)$. Run $z_1 \leftarrow f_1(z_0), z_2 \leftarrow f_2(z_1), \dots, z_k \leftarrow f_k(z_{k-1})$.
3. If z_k is True, let Claim'' be the Claim' transaction with z_0 added as part of the transaction witness. $\text{Claim}'' \xrightarrow{\tau} \mathcal{L}$.
4. Output π .

E.3 Verify($\mathcal{R}, Y, w$) $\rightarrow \{\text{True}, \text{False}\}$:

Challenger:

Upon $\text{Claim}'' \xleftarrow{\tau_1 \leq \tau + u} \mathcal{L}$

1. $(f_1, \dots, f_k) \leftarrow \text{Chunk}(\text{SNARK.Vrfy}, k)$.
2. Read z_0 from the witness of the Claim'' transaction.
3. Run $z'_1 \leftarrow f_1(z_0), z'_2 \leftarrow f_2(z'_1), \dots, z'_k \leftarrow f_k(z'_{k-1})$.
4. If $z'_k = \text{False}$, Challenge $\xrightarrow{\tau_1} \mathcal{L}$.

Upon $\text{Assert}' \xleftarrow{\tau_3 \leq \tau + 3u} \mathcal{L}$:

1. Read $z_0, \dots, z_k$ from the witness of the Assert' transaction.
2. If there is a z_i such that $z_i \neq f_i(z_{i-1})$, let $\text{Disprove}'$ be the Disprove transaction with $z_{i-1}, z_i, c_{z_{i-1}}, c_{z_i}$ added as part of the transaction witness. $\text{Disprove}' \xrightarrow{\tau_3} \mathcal{L}$. Output **False**.
3. If z_k is False, let $\text{Disprove}'$ be the Disprove transaction with z_k, c_{z_k} added as part of the transaction witness. $\text{Disprove}' \xrightarrow{\tau_3} \mathcal{L}$. Output **False**.

Prover :

After round $\tau + \Delta_B$ and Challenge $\notin \mathcal{L}$:

1. $\text{PayoutOptimistic} \xleftarrow{\tau_4 > \tau + \Delta_B} \mathcal{L}$.
2. Output **True**.

Upon Challenge $\xleftarrow{\tau_2 \leq \tau + 2u} \mathcal{L}$:

1. Let Assert' be the Assert transaction with $z_0, c_{z_0}, \dots, z_k, c_{z_k}$ added as part of the transaction witness. $\text{Assert}' \xrightarrow{\tau_2} \mathcal{L}$.

After round $\tau_2 + \Delta_A$ and Disprove $\notin \mathcal{L}$:

1. $\text{Payout} \xleftarrow{\tau_5 > \tau_2 + \Delta_A} \mathcal{L}$.
2. Output **True**.

F Proofs of BITVM2-CORE

In this section, we will prove that the BITVM2-CORE protocol presented in Appendix E satisfies completeness and soundness as defined in Definition D.4 and Definition D.5, respectively. For both security properties, we define a game between a challenger and the adversary (either the prover is the challenger and any verifier is the adversary, or vice versa). Then, we show a reduction from winning one game to breaking either one of the ledger protocol properties or the completeness or the soundness of the SNARK system that we use.

During a game execution, the challenger and the adversary may interact, potentially multiple times, with the robust ledger protocol $\Pi_{\mathcal{L}}(\cdot)$, i.e., a ledger protocol that guarantees safety (see Definition D.1) and liveness (see Definition D.2). The ledger protocol exposes two functionalities: READ, which checks whether a transaction is included in the ledger (i.e., it is confirmed by all honest parties), and WRITE, which attempts to include a transaction in the ledger (i.e., all honest parties will report the transaction). More specifically:

- **READ(tx)**: when **READ** is called at time t_{cur} , it returns 1 if the transaction tx has been written to the ledger by that time. Otherwise it returns 0.
- **WRITE(tx)**: when **WRITE** is invoked at time t_{cur} , it attempts write the transaction tx to the ledger at time t_{cur} .

Before we define the games, we need to lay out some preliminary steps. First, in Algorithm E.1, the **Chunk** subroutine is used. The **Chunk** algorithm breaks a Bitcoin script program into k chunks. In the following lemma, we prove that running these chunks sequentially is equivalent to running the original program.

Lemma F.1. *Let $f : X \rightarrow X$ be a function computed by a Bitcoin script program $\Pi := (i_1, \dots, i_l)$. Let Π be partitioned into k consecutive segments of equal length: $\Pi = \Pi_1 || \Pi_2 || \dots || \Pi_k$, where $||$ denotes the concatenation operator. Each $\Pi_j := (i_{(j-1)k+1}, \dots, i_{jk})$ computes a function $f_j : X \rightarrow X$. Then for all $x \in X$,*

$$f(x) = f_k \circ f_{k-1} \circ \dots \circ f_1(x),$$

where $\circ$ denotes the function composition operator.

Proof. Let us model the semantics of Bitcoin Script programs as functions over stack states. Let $\text{Eval}_\Pi : \text{Stack} \rightarrow \text{Stack}$ denote the deterministic execution of program Π on the stack. Let $x \in X$ be an input encoded as an initial stack state $s_0 := \text{push}(x)$, where **push** is the algorithm that pushes x on the (initially empty) stack. Let $\text{Eval}_\Pi(s_0) = s_k$. Define $f(x) := \text{decode}(s_k)$, where **decode** takes as input a stack state and returns an output that belongs to X . Define

$$s_1 := \text{Eval}_{\Pi_1}(s_0), s_2 := \text{Eval}_{\Pi_2}(s_1), \\ \dots, s_k := \text{Eval}_{\Pi_k}(s_{k-1}).$$

Since $\Pi = \Pi_1 || \Pi_2 || \dots || \Pi_k$, we have that $\text{Eval}_\Pi(s_0) = \text{Eval}_{\Pi_k} \circ \dots \circ \text{Eval}_{\Pi_1}(s_0) = s_k$. Define each $f_j : X \rightarrow X$ as $f_j(x) := \text{decode} \circ \text{Eval}_{\Pi_j} \circ \text{push}(x)$. It follows that

$$f(x) = \text{decode}(s_k) = f_k \circ f_{k-1} \circ \dots \circ f_1(x),$$

which concludes the proof. $\square$

Moreover, we make a key observation that will be crucial in the following security analysis: the adversary cannot interact in an arbitrary way with the ledger protocol through its functionalities, but it is limited by a restrictive covenant emulation system. In our protocol, during the setup phase, the prover and a set of signers collaborate to create a transaction set whose spending conditions induce a restrictive CES, as shown in Lemma A.1. The adversary cannot write a transaction that creates an output that is not in the covenant tree, except with a negligible probability of success.

Below, we prove Lemma A.1.

Proof of Lemma A.1. Suppose, by contradiction, that there is a PPT adversary $\mathcal{A}$ that violates the restrictiveness property. Namely, there is a ledger protocol instance I and a round r such that, with non-negligible probability, $\mathcal{A}$ manages to publish on the ledger a transaction Tx' such that Tx' is in the spending path of the Claim transaction but not in the spending path of neither Challenge, Disprove, Payout, or PayoutOptimistic. There are two ways this event can occur:

- The adversary generates a transaction skeleton Tx' such that, without loss of generality, Tx' spends Claim's output 0. The adversary sends it to the signers, and they all sign (i.e., add their signature to the witness of Tx'). Then, after the timelock Δ_B (cf. Section 3), $\mathcal{A}$ publishes Tx' on the ledger. In this case, the existential honesty assumption is violated.
- The adversary creates Tx' , that, without loss of generality, unlocks Claim's output 0 and forges valid signatures for one or more signer committee members, thus breaking the EUF-CMA security (cf. [15]) of the signature scheme Σ .

In both cases, an assumption is violated, leading to a contradiction. $\square$

Interaction with the ledger. The verifier has access to the ledger's **READ** and **WRITE** functionalities. The prover may request the verifier to perform read or write operations on the ledger. We allow the adversary to choose a round $t \in \{t_{cur}, \dots, t_{cur} + u\}$ (where t_{cur} is the current round) in which it wants a specific transaction tx to appear on-chain.

Lemma F.2. *Assuming a distributed ledger Π_L that is safe (cf. Definition D.1), **READ** functionality is monotonic; that is, for any transaction tx , if there exists a time t_1 such that $\text{READ}(tx) = 1$ when invoked at time t_1 , then $\text{READ}(tx) = 1$ for all $t_2 \geq t_1$.*

Proof. Assume there exists an execution I in which there is a transaction tx and two rounds t_1 and t_2 , such that $t_1 \leq t_2$, and the following hold: (i) $\text{READ}(tx) = 1$ at time t_1 , and (ii) $\text{READ}(tx) = 0$ at time t_2 .

Thus, there exists an honest node p such that the transaction tx is included in its local ledger view C_1 at time t_1 , but not included in its ledger view C_2 at a later time t_2 . This violates the self-consistency of honest parties' views and therefore the safety property of Π_L . $\square$

Lemma F.3. *Assuming a distributed ledger Π_L that is live(u) (cf. Definition D.2), for every transaction tx that is witnessed by all honest parties in some round t , there exists a round $t' \in \{t, \dots, t + u\}$ such that $\text{WRITE}(tx) = 1$.*

Proof. This follows directly from the liveness property of Π_L , which guarantees that transactions broadcast to all honest

parties are eventually included in the ledger within u rounds. $\square$

Theorem F.4. *If the SNARK has knowledge soundness, the ledger is safe and live, and a set of signers and a set of verifiers with existential honesty, BITVM2-CORE satisfies soundness (Definition D.5).*

Proof. Assuming an adversary $\mathcal{A}$ violating the soundness of the Core BitVM, we will construct an adversary $\mathcal{B}$ that, using $\mathcal{A}$, violates either the soundness of the SNARK, or the properties of the underlying distributed ledger protocol $\Pi_{\mathcal{L}}$. To this end, we describe the BITVM2-CORE execution as a game between a malicious prover and honest challenger in Algorithm 7. Assume an execution where $\mathcal{A}$ violates soundness. Namely, after a successful setup returning the set of transactions $(\mathcal{T}, \mathcal{S})$ for some NP-relation $\hat{\mathcal{R}}$, there exists a statement $Y \in \mathcal{L}_{\hat{\mathcal{R}}}$ such that:

- (i) There is no witness w in the view of $\mathcal{A}$ such that $(Y, w) \in \hat{\mathcal{R}}$.
- (ii) A transaction from the set $\mathcal{S}$ is eventually included in the ledger L .

Now, we discuss all the possible ways this event can occur:

- *PayoutOptimistic is on-chain* (line 24): The condition in line 23 is satisfied only if Challenge is not committed on-chain. This may occur in one of the following cases:

- *WRITE failure:* This occurs if WRITE(Challenge) in round t_c fails (line 10). Since this write was invoked at time t_1 with $t_c \leq t_1 + u \leq t_1 + \Delta_B$, and assuming liveness hold such a failure contradicts Lemma F.3.
- *READ failure:* Here, we assume an execution where READ is not monotonic. That is, WRITE(Challenge) succeeds in round t_c (so READ(Challenge) is successful at time t_c), but READ fails at time $t > t_1 + \Delta_B$ (line 23). However, this contradicts Lemma F.2.
- *SNARK soundness violation:* This case arises if the SNARK verification check in line 8 passes despite the absence of a valid witness, thereby violating the soundness of the SNARK system.

- *Payout is on-chain* (line 31): The condition in line 28 fails when Disprove is not committed on-chain. This can happen in the following cases:

- *WRITE failure:* This occurs if WRITE(Disprove) fails in round t_D (line 19). Since it was invoked at time t_2 and $t_D \leq t_2 + u \leq t_2 + \Delta_A$, such a failure contradicts Lemma F.3.

Algorithm 7 Soundness-Game: Run by the challenger against a PPT and q -bounded adversary $\mathcal{A}$. If the game outputs 1, the adversary wins.

```

1:  $crs \leftarrow \text{SNARK.Setup}(R)$ 
2:  $(T, S) \leftarrow \text{Setup}(1^\lambda, \mathcal{R})$ 
3:  $\text{Claim}(Y, \pi), t_1 \leftarrow \mathcal{A}(T)^{\text{TIME}()}$   $\triangleright \mathcal{A}$  requests to write
   on-chain  $\text{Claim}(Y, \pi)$  at time  $t_1$ 
4: if  $\text{READ}(\text{Claim}(Y, \pi)) = 1$  in round  $t_1 - 1$  then
5:   Return
6: end if
7: if  $\text{WRITE}(\text{Claim}(Y, \pi)) = 1$  in round  $t_1$  then
8:   if  $\neg \text{SNARK.Vrfy}(\mathcal{R}, crs, Y, \pi)$  then
9:      $t_C \leftarrow \mathcal{A}^{\text{TIME}(\leq t_1 + \Delta_B)}$ 
10:     $\text{WRITE}(\text{Challenge}(Y, \pi))$  in round  $t_C$ 
11:   end if
12: end if
13: if  $\text{READ}(\text{Challenge}(Y, \pi)) = 1$  in round  $t_C$  then
14:    $\text{Assert}(Y, \pi), t_2 \leftarrow A(T)$ 
15:    $\text{WRITE}(\text{Assert}(Y, \pi))$  in round  $t_2$ 
16: end if
17: if  $\text{READ}(\text{Assert}(Y, \pi)) = 1$  in round  $t_2$  then
18:    $t_D \leftarrow \mathcal{A}^{\text{TIME}(\leq t_2 + \Delta_A)}$ 
19:    $\text{WRITE}(\text{Disprove}(Y, \pi))$  in round  $t_D$ 
20: end if

21: At time  $t_1 + \Delta_B$ :
22:    $\text{PayoutOptimistic}(Y, \pi), t \leftarrow A(S)^{\text{TIME}(> t_1 + \Delta_B)}$ 
23:   if  $(\text{READ}(\text{Challenge}(Y, \pi)) = 0 \wedge \text{Write}(\text{PayoutOptimistic}(Y)) = 1)$  in round  $t$  then
24:     Output 1
25:   end if

26: At time  $t_2 + \Delta_A$ :
27:    $\text{Payout}(Y, \pi), t \leftarrow A(S)^{\text{TIME}(> t_2 + \Delta_A)}$ 
28:   if  $\text{READ}(\text{Disprove}(Y, \pi)) = 1$  in round  $t$  then
29:     Output 0
30:   else if  $\text{Write}(\text{Payout}(Y, \pi)) = 1$  in round  $t$ 
31:     Output 1
32:   else Output 0
33:   end if

```

- *READ failure*: Again, we assume an execution where `READ` is non monotonic. Specifically, we assume that `WRITE(Disprove)` is successful in the round t_D (so `READ` is successful in round t_D), but `READ` fails in round $t > t_2 + \Delta_A$ (line 28) contradicting Lemma F.2.

□

Theorem F.5. *Assuming the SNARK is complete, the ledger is safe and live, and a set of signers with existential honesty, BITVM2-CORE satisfies completeness (Definition D.4).*

Proof. Assume, for contradiction, the adversary $\mathcal{A}$ violates the completeness of the Core BitVM. We will construct an adversary $\mathcal{B}$ that, using $\mathcal{A}$, breaks either the completeness of the SNARK, or the properties of the underlying distributed ledger protocol Π_L . To argue that, we present the BITVM2-CORE execution as a game between an honest prover and a malicious challenger in Algorithm 8.

Assume an execution in which $\mathcal{A}$ succeeds in violating completeness. That is, after a successful setup returning a set of transactions $(\mathcal{T}, \mathcal{S})$ for some NP-relation $\mathcal{R}$, there exists a statement $Y \in \mathcal{L}_{\mathcal{R}}$ such that:

- (i) There exists a witness w in the prover’s view such that $(Y, w) \in \mathcal{R}$.
- (ii) None of the transaction in the set $\mathcal{S}$ gets eventually included in the ledger $\mathcal{L}$.

We now analyze the possible causes of this violation:

- *WRITE(Claim(Y, π)) failure* (line 13): This happens with a negligible probability in the security parameter λ .
- *Assert(Y, π) is not on-chain in the end of the execution*: This can happen either if `WRITE(Assert( $Y, \pi$ ))` fails in round t_2 (line 17), or if `READ` is not monotonic, i.e., `READ(Assert( $Y, \pi$ ))` is successful in round t_2 but fails in the end of the execution. None of these scenarios can occur according to Lemmas F.2 and F.3.
- *READ(Disprove(Y, π)) is successful* (line 34): This occurs if the SNARK verification check in line 21 fails, despite the prover possessing a valid witness w such that $(Y, w) \in \mathcal{R}$. In this case, we can construct an adversary $\mathcal{B}$ that breaks the completeness of the SNARK scheme.

□

G Light Client Analysis

In this section, we first expose the necessary preliminaries from the Bitcoin backbone protocol analysis [13], then introduce the new definitions required for our analysis, and finally prove security for our on-chain light client protocol.

Algorithm 8 Completeness-Game: Run by the challenger against a PPT and q-bounded adversary $\mathcal{A}$. If the game outputs 1, the adversary wins.

```

1: Input:  $\lambda$ : security parameter, relation  $\mathcal{R}$ 

2:  $crs \leftarrow \text{SNARK.Setup}(R)$ 
3:  $(T, S) \leftarrow \text{Setup}(1^\lambda, \mathcal{R})$ 
4:  $\text{Claim}(Y, \pi), t_1 \leftarrow \mathcal{A}(T)^{\text{TIME}()}$   $\triangleright \mathcal{A}$  requests to write
   on-chain  $\text{Claim}(Y, \pi)$  at time  $t_1$ 
5: if WRITE(Claim( $Y, \pi$ )) = 1 in round  $t_1$  then
6:    $t_C \leftarrow \mathcal{A}$   $\triangleright \mathcal{A}$  requests to write on-chain
   Challenge( $Y, \pi$ ) at time  $t_C$ 
7:   if  $t_C \leq t_1 + \Delta_B$  then
8:     WRITE(Challenge( $Y, \pi$ )) in round  $t_C$ 
9:   else
10:     $t \leftarrow \mathcal{A}^{\text{TIME}(\leq t_1 + \Delta_B + 1)}$ 
11:    WRITE(PayoutOptimistic( $Y, \pi$ )) in round  $t$ 
12:   end if
13: else Output 1
14: end if
15: if READ(Challenge( $Y, \pi$ )) = 1 in round  $t_C$  then
16:    $t_2 \leftarrow \mathcal{A}^{\text{TIME}(\leq t_1 + \Delta_C)}$ 
17:   WRITE(Assert( $Y, \pi$ )) in round  $t_2$ 
18: end if
19: if READ(Assert( $Y, \pi$ )) = 1 in round  $t_2$  then
20:    $\text{Disprove}(Y, \pi), t_D \leftarrow \mathcal{A}(T)^{\text{TIME}(\leq t_2 + \Delta_A)}$ 
21:   if  $\neg \text{SNARK.Vrfy}(\mathcal{R}, crs, Y, \pi)$  then
22:     WRITE(Disprove( $Y, \pi$ )) in round  $t_D$ 
23:   else
24:     $t_3 \leftarrow \mathcal{A}(T)^{\text{TIME}(\leq t_2 + \Delta_A + 1)}$ 
25:    WRITE(Payout( $Y, \pi$ )) in round  $t_3$ 
26:   end if
27: end if

28: At time  $t_1 + \Delta_B + 1$ :
29:   if READ(PayoutOptimistic( $Y, \pi$ )) = 1 then
30:     Output 0
31:   end if

32: At time  $t_2 + \Delta_A$ :
33:   if READ(Disprove( $Y, \pi$ )) = 1 then
34:     Output 1
35:   end if

36: At time  $t_2 + \Delta_A + 1$ :
37:   if READ(Payout( $Y, \pi$ )) = 1
38:     Output 0
39:   end if

```

G.1 Formal properties and definitions

The properties of blockchain protocols defined in the backbone model are presented below. Such properties are defined as predicates over the random variable $\text{view}_{\Pi,A,Z}^{t,n}$ by quantifying over all possible adversaries A and environments Z that are polynomially bounded. Note that blockchain protocols typically satisfy properties with a small probability of error in a security parameter κ (or others). The probability space is determined by random queries to the random oracle functionality and by the private coins of all interactive Turing machine instances.

The q -bounded setting

Parties belonging to $\mathcal{P}$, which we call consensus nodes, and the adversary $\mathcal{A}$ have access to an ITM, which we name *random oracle*, that works as follows. Let $H(\cdot) : \{0,1\}^* \rightarrow \{0,1\}^k$ be a hash function. The random oracle maintains a table of pairs $(x,y) \in \{0,1\}^* \times \{0,1\}^k$. When a party invokes the random oracle giving as input a string $x \in \{0,1\}^*$, the random oracle outputs the string y to the caller. If x has not been queried before, the random oracle returns a value y selected at random from $\{0,1\}^k$ and stores the value (x,y) in its table.

In each round, each party is allowed to query the random oracle subroutine at most q times. Consequently, in each round, the adversary is allowed to ask at most $t' \cdot q$ queries, where $t' \leq t$, the maximum number of parties that the adversary can corrupt during a protocol execution. In the following, we refer to the setting where the parties and the adversary are restricted in the number of queries per round as the *q -bounded synchronous setting*.

Definition G.1 (Common Prefix Property [13]). The common prefix property Q_{cp} with parameter $k \in \mathbb{N}$ states that for any pair of honest players P_1, P_2 adopting the chains C_1, C_2 at rounds $r_1 \leq r_2$ in $\text{view}_{\Pi,A,Z}^{t,n}$ respectively, it holds that $C_1^{[k]} \leq C_2$.

Definition G.2 (Chain Quality Property [13]). The chain quality property Q_{cq} with parameters $\mu \in \mathbb{R}$ and $\ell \in \mathbb{N}$ states that for any honest party P with chain C in $\text{view}_{\Pi,A,Z}^{t,n}$, it holds that for any ℓ consecutive blocks of C , the ratio of honest blocks is at least μ .

Definition G.3 (Chain Growth Property [13]). The chain growth property Q_{cg} with parameters $\tau \in \mathbb{R}$ and $s \in \mathbb{N}$ states that for any honest party P that has a chain C in $\text{view}_{\Pi,A,Z}^{t,n}$, it holds that after any s consecutive rounds, it adopts a chain that is at least $\tau \cdot s$ blocks longer than C .

Closely following [13], we will call a query $q \in \mathbb{N}$ of a party successful if it returns a valid solution to the PoW. For each round i , $j \in [q]$, and $k \in [t]$, we define Boolean random variables X_i, Y_i , and Z_{ijk} as follows. If at round i an honest

party obtains a PoW, then $X_i = 1$, otherwise $X_i = 0$. If at round i exactly one honest party obtains a PoW, then $Y_i = 1$, otherwise $Y_i = 0$. Regarding the adversary, if at round i , the j -th query of the k -th corrupted party is successful, then $Z_{ijk} = 1$, otherwise $Z_{ijk} = 0$. Define also $Z_i = \sum_{k=1}^t \sum_{j=1}^q Z_{ijk}$. For a set of rounds S , let $X(S) = \sum_{r \in S} X_r$ and similarly define $Y(S)$ and $Z(S)$. Further, if $X_i = 1$, we call i a successful round and if $Y_i = 1$, a uniquely successful round. We denote with f the probability that at least one honest party succeeds in finding a PoW in a round.

Definition G.4 (Typical Execution [13]). An execution is (ϵ, λ) -typical (or just typical), for $\epsilon \in (0, 1)$ and integer $\lambda \geq 2/f$, if, for any set S of at least λ consecutive rounds, the following hold.

- (a) $(1 - \epsilon)\mathbb{E}[X(S)] < X(S) < (1 + \epsilon)\mathbb{E}[X(S)]$ and $(1 - \epsilon)\mathbb{E}[Y(S)] < Y(S)$.
- (b) $Z(S) < \mathbb{E}[Z(S)] + \epsilon\mathbb{E}[X(S)]$.
- (c) No insertions, no copies, and no predictions occurred.

Let n be the number of consensus nodes, out of which t are controlled by the adversary. Let Q be an upper bound on the number of computation or verification queries to the random oracle. Let L be the total number of rounds in the execution, and λ, κ security parameters. Finally, we denote with v the min-entropy of the value that the miner attempts to insert in the chain.

Theorem G.1 (Theorem 4.5 in [13]). *An execution is not typical with probability less than*

$$\epsilon_{\text{typ}} = 4L^2 e^{-\Omega(\epsilon^2 \lambda f)} + 3Q^2 2^{-\kappa} + [(n-t)L]^2 2^{-v}.$$

Lemma G.2 (Lemma 4.6 in [13]). *The following hold for any set S of at least λ consecutive rounds in a typical execution. For $S = \{i : r < i < s\}$ and $S' = \{i : r \leq i \leq s\}$, $Z(S') < Y(S)$.*

Lemma G.3 (Lemma 4.8 in [13], (aka Patience Lemma)). *In a typical execution, any $k \geq 2\lambda f$ consecutive blocks of a chain have been computed in more than $\frac{k}{2f}$ consecutive rounds.*

G.2 Light client security definitions

Now, recall the definition of an admissible block [7].

Definition G.5 ((u, k) -Admissible Block at r). Consider $u, k \in \mathbb{N}$. A block B that, at round r , fulfils the following properties is an *admissible block at r* :

- **Safety:** $B \in C_{r+u}^{\cup}[-k]$
- **Liveness:** $B \notin C_r^{\cap}[-k]$

G.4 Liveness proof

Proof. Liveness follows from the fact that u is defined as the time it takes for the honest chain to grow by at least $m + 1$ blocks. Since $B := C[-m - k]$, and chain has grown by at least $m + 1$ blocks between r and $r + u$, B was not part of $C_r^{\cup}[-k]$. $\square$

H Bridge security analysis

In this section, we will prove that our wrapped asset bridge protocol satisfies *bridge safety* and *bridge liveness*, both for peg-in and peg-out (see Definitions D.12 to D.14), as well as *bridge operator safety*.

H.1 Peg-in bridge

The peg-in bridge $\Lambda_{R_1}^{\text{PegIn}}$ is parameterized by the relation $R_1(\text{id}) = (\{\text{PegIn}(\text{id})\}, \{\text{Mint}(\text{id})\})$, which binds lock transactions on $\mathcal{L}_A$ to minting transactions on $\mathcal{L}_B$. As discussed in Section 5, the safety and liveness of $\Lambda_{R_1}^{\text{PegIn}}$ depend on the sidechain system, assuming that the source ledger $\mathcal{L}_A$ remains safe and live; satisfying Theorem 6.5 and Theorem 6.4.

H.2 Peg-out bridge

We now prove that the peg-out bridge $\Lambda_{R_2}^{\text{PegOut}}$ satisfies both *bridge safety* and *bridge liveness*. The bridge is parameterized by the relation

$$R_2(\text{id}) = \left(\begin{array}{c} \{\text{PegOut}(\text{id})\}, \\ \{\text{Burn}(\text{id})\} \end{array} \right),$$

which binds burn transactions on $\mathcal{L}_B$ to corresponding payout transactions on $\mathcal{L}_A$.

In addition, we show that the bridge satisfies *bridge operator safety*: when a PegOut is initiated by an honest operator, the operator will eventually be able to successfully claim their funds.

Theorem H.1 (Peg-out Bridge Safety). *The peg-out bridge $\Lambda_{R_2}^{\text{PegOut}}$ satisfies safety with parameter u_s assuming at least one honest signer post-setup, and $\mathcal{L}_B$ satisfies safety.*

Proof. Assume, for contradiction, that a transaction $\text{tx}' \in \text{PegOut}(\text{id})$ is included in $\mathcal{L}_A$ at some round r_2 , but no transaction $\text{tx} \in \text{Burn}(\text{id})$ appears in $\mathcal{L}_B$ at round $r_1 \geq r_2 - u_s$.

The scenario can occur only for one of the following reasons:

- *Setup failure.* A setup failure occurs if the first input of $\text{tx}' \in \text{PegOut}(\text{id})$, which is supposed to be fixed, exists without a corresponding $\text{PegIn}(\text{id})$ at the time the $\text{PegOut}(\text{id})$ transaction was posted on-chain. This implies that the signer committee incorrectly authorized

a tx' transaction that violates this constraint. Such a failure contradicts either the assumption that committee members act honestly during the setup phase, or the assumption of existential honesty after setup, which ensures that at least one honest signer deletes their signing key and cannot be corrupted dynamically. If this assumption is violated, a fully corrupted committee could effectively re-run the setup phase and authorize arbitrary, invalid transactions.

- *Safety failure of $\mathcal{L}_B$.* A transaction $\text{tx} \in \text{Burn}(\text{id})$ was included in $\mathcal{L}_B$ at some round $r < r_1$, and an operator p initiated the peg-out process, resulting in (tx', p) appearing in $\mathcal{L}_A$ at round r_2 . However, tx does not persist in $\mathcal{L}_B$ for some round after r_1 . Such a failure contradicts the *self-consistency* of honest parties' views and therefore violates the *safety* property of $\mathcal{L}_B$. $\square$

Theorem H.2 (Peg-out Bridge Liveness). *The peg-out bridge $\Lambda_{R_2}^{\text{PegOut}}$ satisfies liveness with parameter u_ℓ assuming all signers are honest during setup while at least one remains honest post-setup, $\mathcal{L}_A$ is safe and live, and an honest operator monitors $\mathcal{L}_B$.*

Proof. Let $\text{Burn}(\text{id})$ be included in $\mathcal{L}_B$ at round r_1 . By the bridge protocol, an honest operator observing this event is required to initiate the peg-out process at round r_1 by posting a transaction $\text{tx}' \in \text{PegOut}(\text{id})$ to $\mathcal{L}_A$. This should result in some transaction (tx', p) , for some operator p , appearing in $\mathcal{L}_A$ at round $r_1 + u$ where u is the liveness parameter of $\mathcal{L}_A$. This process can fail only in the following ways:

- *Setup failure.* The signer committee failed to construct the transactions as specified during the setup, making it infeasible for an honest operator to post a transaction $\text{tx}' \in \text{PegOut}(\text{id})$. This violates the assumption that all committee members behave honestly during the setup phase.
- *Liveness failure of $\mathcal{L}_A$.* Any honest operator p that submits a transaction (tx', p) to $\mathcal{L}_A$ fails to have it included within u rounds. This contradicts the liveness property of $\mathcal{L}_A$.
- *Safety failure of $\mathcal{L}_A$.* A transaction (tx', p) , for some operator p , was included in $\mathcal{L}_A$ at round $r_1 + u_\ell$. However, the transaction does not persist in $\mathcal{L}_A$ at some round after $r_1 + u_\ell$. Such a failure contradicts the *self-consistency* of honest parties' views and therefore violates the *safety* property of $\mathcal{L}_A$. $\square$

Theorem H.3 (Bridge Operator Safety). *The bridge satisfies operator safety with recovery parameter u_Λ , assuming the*

signer committee is honest during setup and one member remains honest after, an honest challenger exists, both ledgers are live and safe, the light client is secure, and BITVM2-CORE is complete and sound.

Proof. Assume, for contradiction, that the transaction (tx, p_i) with $tx \in \text{PegOut}(\text{id})$ appears in the ledger of all honest parties in $\mathcal{L}_A$ at round r_1 , for some honest operator p_i . However, no transaction (tx', p_i) , where $tx' \in \text{Payout}(\text{id}), \text{PayoutOptimistic}(\text{id})$, appears in $\mathcal{L}_A$ at round $r_1 + u_\Lambda$.

At most one transaction $tx' \in \{\text{Payout}(\text{id}), \text{PayoutOptimistic}(\text{id})\}$ can be published. This is because each such transaction has as input a transaction in $\text{PegIn}(\text{id})$ output, which can be spent only once in the UTXO model. Based on this observation, we distinguish the following cases:

Case 1: There exists a transaction (tx', p_j) in $\mathcal{L}_A$ at some round $r' < r_1 + u_\Lambda$, where $p_j \neq p_i$. The transaction (tx', p_j) can appear in $\mathcal{L}_A$ without a corresponding (tx, p_j) for the following reasons:

- *Signer committee violation.* The signer committee incorrectly authorized some transaction related to the BITVM2-CORE instance during setup—e.g., by presigning a transaction that enables a dishonest operator to extract funds from the bridge arbitrarily. This violates either the assumption that the committee behaves honestly during setup, or the assumption of existential honesty after setup which guarantees that at least one honest signer cannot be corrupted and deletes their signing key. If the latter assumption fails, a newly corrupted committee could effectively recreate the setup phase and authorize arbitrary, malicious transactions as if they had been honest from the beginning.
- *Light client liveness failure.* Suppose p_j knows a pair (ϕ, w) such that $(\phi, w) \in R := \{(\phi, w) : \Pi(\phi, w) = \text{True}\}$ (cf. Algorithm 3), even though no transaction (tx, p_j) appears in $\mathcal{L}_A$ at round r' . Then p_j must have produced a valid witness w for the statement ϕ , where $w[-m-k]$ includes (tx, p_j) and also satisfies the light client relation $R := \{(\phi, w) : \Pi_{lc}(\phi, w) = \text{True}\}$. That means that $w[-m-k]$ is an admissible block. However, since this transaction does not appear in $\mathcal{L}_A$ at round r' , this contradicts the safety of the light client (Definition G.5). By Appendix G.3, such a violation can occur only with negligible probability.
- *BitVM Core soundness failure.* Suppose none of the above scenarios occur, and thus p_j does not know any pair (ϕ, w) that satisfies the relation $R := \{(\phi, w) : \Pi(\phi, w) = \text{True}\}$. Yet one of the transactions (tx', p_j) for $tx' \in \{\text{Payout}(\text{id}), \text{PayoutOptimistic}(\text{id})\}$ is committed to $\mathcal{L}_A$. This violates the soundness of BitVM. Assuming a correct setup, at least one committee member

is honest, and an honest challenger participates during execution, such a failure can occur only with negligible probability, as shown in Theorem 6.1.

Case 2: No transaction tx' appears on chain for all rounds $r \leq r_1 + u_\Lambda$.

- *Setup failure.* The signer committee did not construct the transactions as specified by the Setup phase of the BITVM2-CORE instance, making it infeasible for an honest operator to reclaim its money. This violates the assumption that all committee members act honestly during the setup phase.
- *Light client liveness failure.* In this scenario, although (tx, p_i) appears in $\mathcal{L}_A$ at round r_1 , p_i does not yet know a pair (ϕ, w) such that $(\phi, w) \in R := \{(\phi, w) : \Pi(\phi, w) = \text{True}\}$ (see Algorithm 3). However, we argue that once the block B containing (tx, p_i) becomes $m+k$ blocks deep in the chain p_i can construct a valid pair (ϕ, w) as follows.

In particular, p_i can provide the ledger $\mathcal{L}_A$ as the witness w , and demonstrate that $B = w[-m-k]$ is an admissible block. Since B includes the transaction (tx, p_i) and satisfies the light client relation $R := \{(\phi, w) : \Pi_{lc}(\phi, w) = \text{True}\}$ with overwhelming probability by the light client liveness property (Definition G.5, Appendix G.4), the pair (ϕ, w) is valid.

- *BitVM Core completeness failure:* Suppose none of the previously considered scenarios apply, and thus p_i knows a pair (ϕ, w) satisfying the relation $R := \{(\phi, w) : \Pi(\phi, w) = \text{True}\}$. Still, no transaction $tx' \in \{\text{Payout}(\text{id}), \text{PayoutOptimistic}(\text{id})\}$ is committed on-chain within u_c rounds. This scenario contradicts the completeness of BitVM. However, under correct setup and assuming the operator is honest, such a failure can occur only with negligible probability, as guaranteed by Theorem 6.2.

In both cases, we reach a contradiction, concluding that if $(\text{PegOut}(\text{id}), p_i)$ appears on $\mathcal{L}_A$, then a corresponding transaction (tx', p_i) with $tx' \in \{\text{Payout}(\text{id}), \text{PayoutOptimistic}(\text{id})\}$ must appear on $\mathcal{L}_A$ within $u_\Lambda = u_{\mathcal{L}_A} + u_c$ rounds (it takes $u_{\mathcal{L}_A}$ rounds to submit `Claim` on-chain, and an additional u_c rounds for the BITVM2-CORE instance to complete once the `Claim` is posted.). $\square$